LLMOps: From Laptop to Production

A complete end-to-end course on building, shipping and operating ML, DL and LLM-powered applications

How to use this course

This course walks the full path of an AI application: an idea in a notebook on your laptop, through a clean codebase, data pipelines, experiments, evaluation, containers, serving, CI/CD, monitoring, security and cost control, to a system that real users depend on. It is pitched at an engineer who has built a few AI projects and now wants to understand how production systems are actually designed.

It has 22 modules in 5 parts:

- **Part A, Foundations (Modules 1-5):** the big picture, local setup, data, experiments, LLM application patterns
- **Part B, Quality and Packaging (Modules 6-8):** evaluation, containerization and APIs, model serving and inference
- **Part C, Operations (Modules 9-13):** CI/CD and infrastructure, observability, guardrails and security, cost and latency, reliability
- **Part D, System Design and Practice (Modules 14-16):** reference architectures, a design framework, governance and feedback loops, capstone projects and a 12-week roadmap
- **Part E, Deep-Dive Tracks (Modules 17-22):** DevOps and backend engineering, MLOps tooling, LLMOps specialization, distributed systems and production design, monitoring and reliability, plus a combined learning path

Every module follows the same shape: **concepts, why it matters, how it is done, common tools, pitfalls, exercise**. Tools change every few months; the concepts underneath do not. Learn the concepts first, then pick tools.

PART A: FOUNDATIONS

Module 1: The Big Picture

1.1 What 'production' actually means

A model in a notebook is a *demo*. A production system is one where:

- Many users can call it at once, reliably, at acceptable latency
- Its behavior is measured, so you know when it gets worse
- Changes ship safely and can be rolled back in minutes
- Cost per request is known and bounded
- Data, code, models and prompts are versioned, so any result can be reproduced
- Failures are handled gracefully and security risks are controlled

The gap between demo and production is usually 80% of the work, and almost none of it is modeling.

1.2 The end-to-end lifecycle

Every AI application, classical ML or LLM, runs through the same loop:

```
Problem framing -> Data -> Experiment/Develop -> Evaluate -> Package
      ^                                                        |
      |                                                        v
Feedback & retraining <- Monitor <- Operate <- Deploy <- Release gates
```

The important word is *loop*. Production AI is never finished: data drifts, users behave differently than you expected, models and providers update, and requirements change. LLMOps is the discipline of making that loop fast, safe and cheap.

1.3 DevOps vs MLOps vs LLMOps

- **DevOps:** ship code reliably (CI/CD, infra as code, monitoring)
- **MLOps:** DevOps plus data and model lifecycle (data versioning, training pipelines, model registry, drift monitoring)
- **LLMOps:** MLOps adapted to foundation models, where you often do not train anything. The new artifacts are prompts, retrieval indexes, tools and agent logic. The new problems are non-deterministic outputs, evaluation without ground truth, token-

based cost, and security issues like prompt injection

Aspect	Classical ML	LLM applications
Main artifact	Trained model	Prompt + context + model choice + tools
Training	You train	Mostly none, sometimes fine-tune
Output	Label / number, deterministic	Free text, non-deterministic
Evaluation	Accuracy, F1, RMSE on labeled test set	Rubrics, LLM judges, human review, task success
Cost driver	Training compute, serving compute	Tokens per request
Latency	Milliseconds	Seconds (streaming matters)
Failure mode	Wrong prediction, drift	Hallucination, injection, refusal, format break
Versioning	Code, data, model weights	Plus prompts, retrieval index, provider model version

1.4 The layers of a production AI system

Think of the system as layers. Most design discussions are really about which layer a problem belongs to.

1. **Client layer:** web, mobile, API consumers, chat UI
2. **Edge and API layer:** load balancer, API gateway, authentication, rate limiting
3. **Application layer:** business logic, orchestration (RAG pipeline, agent loop), prompt assembly
4. **Model layer:** hosted APIs (Anthropic, OpenAI, etc.) or self-hosted models behind an inference server
5. **Data layer:** relational DB, object storage, vector DB, cache, queues, feature store
6. **Platform layer:** containers, Kubernetes or serverless, networking, secrets, IaC
7. **Ops layer:** CI/CD, evaluation, observability, alerting, cost tracking
8. **Governance layer:** access control, audit logs, data privacy, compliance

1.5 MLOps maturity levels

- **Level 0, manual:** notebooks, manual steps, model handed over as a file. Fine for research, dangerous for production
- **Level 1, pipeline automation:** training and data steps run as automated pipelines, experiments are tracked, models are registered
- **Level 2, CI/CD for ML:** code, pipelines and models are tested and deployed automatically, with evaluation gates
- **Level 3, continuous learning:** production feedback automatically feeds evaluation sets, retraining and prompt improvements, with monitoring closing the loop

Aim to reach Level 2 for any serious project, and know what Level 3 would add.

1.6 Core principles to remember

1. **Everything is versioned:** code, data, config, prompts, models, indexes
2. **Everything is reproducible:** same inputs and versions give the same pipeline run
3. **Evaluate before you ship, monitor after:** evals are the unit tests of AI
4. **Make the smallest thing work end to end first,** then improve the weakest stage
5. **Separate concerns:** config from code, orchestration from model calls, serving from training
6. **Design for failure:** every external call can be slow, wrong or down
7. **Measure cost and latency from day one**

Exercise 1: Pick an AI app you use (a support chatbot, a code assistant). Draw its layers from section 1.4 and mark where you think the model lives, where data lives, and three places it could fail.

Module 2: Local Development Environment and Project Structure

2.1 Why structure matters

Production code is read far more than written, and run by people (and pipelines) that are not you. The goal is a repo where a new engineer can clone, run one command, and have it working.

2.2 A solid repository layout

```
my-ai-app/
  pyproject.toml          # dependencies, tool config
  uv.lock                 # pinned, reproducible versions
  README.md
  Makefile                # make test, make lint, make run
  .env.example            # documented config, no real secrets
  Dockerfile
  docker-compose.yml      # app + db + vector store for local dev
  src/myapp/
    api/                  # FastAPI routes, request/response schemas
    core/                 # settings, logging, errors
    services/             # business logic (rag.py, agent.py)
    llm/                  # provider clients, prompt templates, parsers
    data/                 # loaders, chunkers, embedding jobs
    models/               # training / fine-tuning code if any
  prompts/                # versioned prompt files (yaml/jinja)
  evals/                  # datasets, graders, eval runner
  tests/                  # unit and integration tests
  notebooks/              # exploration only, never imported
  infra/                  # Terraform / Helm / k8s manifests
  .github/workflows/      # CI/CD pipelines
```

Key ideas: `src/` layout avoids import accidents, notebooks are for exploring only, prompts and evals live in the repo as first-class artifacts.

2.3 Environment management

- **Pin Python version** (`.python-version`) and **lock dependencies** (uv, Poetry, pip-tools). A lockfile means CI, your laptop and production run identical versions
- Separate dependency groups: runtime, dev, eval, training. Production images should not carry Jupyter or pytest
- Use **dev containers or docker-compose** for services (Postgres, Redis, Qdrant) so local mirrors production
- GPU work: pin CUDA, PyTorch and driver compatibility explicitly; mismatches are the top source of 'works on my machine'

2.4 Configuration and secrets

Follow the **12-factor** rule: config lives in the environment, not in code.

- Use a typed settings class (pydantic-settings) that reads env vars and fails fast on missing values
- `.env` for local only, never committed; commit `.env.example`
- Secrets (API keys, DB passwords) go in a secrets manager (AWS Secrets Manager, GCP Secret Manager, Vault, Kubernetes secrets) in real environments
- Config that differs per environment: model name, endpoints, timeouts, feature flags, log level
- **Never log secrets or full prompts containing user PII**

2.5 From notebook to module

Notebooks are great for exploration and terrible for production. Graduate code like this:

1. Prove the idea in a notebook
2. Extract functions with clear inputs and outputs into `src/`
3. Add type hints and a docstring
4. Write a test that pins the behavior
5. Import the function back into the notebook

2.6 Code quality tooling

- **Formatting and linting:** ruff (fast linter and formatter)
- **Types:** mypy or pyright; typed boundaries (pydantic models) catch bad data early
- **Tests:** pytest; unit tests for pure logic, integration tests against local services
- **pre-commit hooks:** run lint, format, secret scanning (gitleaks) before every commit
- **Git workflow:** short-lived branches, pull requests, required CI checks, conventional commit messages

2.7 Testing AI code (the realistic way)

You cannot unit test 'is the answer good', but you can test everything around it:

- Parsers, chunkers, prompt builders: deterministic, fully unit testable
- LLM calls: mock the client in unit tests; use recorded responses for regression
- Integration tests: real call against a cheap model with loose assertions (valid JSON, contains citation)
- Quality testing is a separate system, covered in Module 6

Exercise 2: Take any notebook project of yours and restructure it into the layout above with a locked environment, typed settings, one passing test and a pre-commit config. Goal: `git clone && make setup && make test` works on a fresh machine.

Module 3: The Data Layer

3.1 Data is the real product

Most production failures trace back to data: wrong, stale, leaked, or different between training and serving. Treat data pipelines with the same rigor as code.

3.2 Data lifecycle

Sources -> Ingestion -> Validation -> Transformation -> Storage -> Versioning -> Serving

- **Ingestion:** batch (scheduled loads from DBs, files, APIs) or streaming (Kafka, Kinesis, Pub/Sub). Choose batch unless freshness demands streaming
- **Validation:** schema checks, null rates, ranges, duplicates, distribution checks (Great Expectations, pandera, Pydantic). Fail the pipeline loudly rather than train on garbage
- **Storage:** raw data is immutable in object storage (S3/GCS) as the source of truth; processed data lives in a warehouse or lake (BigQuery, Snowflake, Parquet on S3, Delta/Iceberg)
- **Versioning:** DVC, lakeFS, or Delta time travel, so a model can always be tied to the exact data it saw

3.3 Classical ML data concerns

- **Train/validation/test splits:** split before any fitting or normalization; for time-dependent data split by time, never randomly
- **Data leakage:** features that contain the answer or future information give great offline metrics and awful production results
- **Training-serving skew:** features computed one way in training and another in production. Fix with a **feature store** (Feast, Tecton, Databricks/Vertex feature stores) or shared transformation code
- **Label quality:** noisy labels cap your ceiling; budget time for labeling guidelines and review

- **Class imbalance, sampling, augmentation** belong in the pipeline, not in ad hoc notebooks

3.4 LLM data concerns: knowledge and RAG

For LLM apps, 'data' is mostly the knowledge you give the model at request time. The ingestion pipeline for RAG:

1. **Collect:** PDFs, wikis, tickets, databases, web pages, code
2. **Parse:** layout-aware extraction (tables, headings, OCR for scans). Bad parsing silently ruins everything downstream
3. **Clean:** remove boilerplate, headers/footers, duplicates; attach metadata (source, date, owner, permissions, version)
4. **Chunk:** split into retrievable units
5. **Embed:** convert chunks to vectors
6. **Index:** store in a vector DB with metadata
7. **Refresh:** keep the index in sync with the source

Chunking strategies:

- Fixed size with overlap (simple baseline, e.g. 300-800 tokens, 10-15% overlap)
- Structure-aware: split on headings, paragraphs, code functions, table rows
- Semantic: split where topic shifts
- Hierarchical (parent-child): retrieve small chunks, feed the larger parent to the model
- Contextual chunking: prepend a short summary of the document or section to each chunk so it makes sense in isolation

Embeddings: pick a model for your language and domain, version it, and remember that **changing the embedding model means re-embedding everything**. Track the embedding model name in the index metadata.

Vector databases: pgvector (great default if you already use Postgres), Qdrant, Weaviate, Pinecone, Milvus, OpenSearch, FAISS (library, not a service). Core concepts:

- ANN indexes (HNSW, IVF) trade a little recall for big speed
- Metadata filtering (tenant, date, document type) is essential for multi-tenant and permissioned data
- **Hybrid search** combines vector similarity with keyword search (BM25) and usually beats either alone
- **Rerankers** (cross-encoders) rescore the top 20-50 results to pick the best 3-8

Freshness and deletion: use incremental updates keyed by document hash, handle deletes (a removed document must disappear from the index), and respect access control at retrieval time, not just ingestion.

3.5 Privacy and compliance in data

- Classify data (public, internal, confidential, PII)
- Mask or remove PII before sending to third-party models unless your agreement and policy allow it
- Record data lineage: where each piece came from and where it went
- Follow retention rules; plan for deletion requests

Exercise 3: Build an ingestion pipeline for 50 documents of your choice: parse, chunk with two strategies, embed, store in pgvector or Qdrant with metadata, and write 10 test queries. Compare which chunking returns better results by eye.

Module 4: Experimentation and Model Development

4.1 Reproducible experiments

An experiment is only useful if you can explain and repeat it. For every run, record:

- Code version (git commit), data version, environment (lockfile hash)
- Hyperparameters and config, random seeds
- Metrics over time, artifacts (model files, plots, sample outputs)
- For LLM experiments: prompt version, model name and version, temperature, retrieval settings

Tools: **MLflow**, **Weights & Biases**, Neptune, Comet, ClearML. Even a disciplined spreadsheet beats memory.

4.2 The model registry

A registry is the handoff point between development and deployment. It stores versioned models with metadata (metrics, data version, owner) and **stages** (staging, production, archived). Deployment pipelines pull 'the production model' from the registry rather than from someone's laptop. For LLM apps, the equivalent registry covers prompts and configs (see Module 5).

4.3 Training pipelines

Turn training into a pipeline of small, testable steps: data pull, validate, featurize, train, evaluate, register. Orchestrators schedule and retry steps and track lineage:

- **Airflow:** battle-tested scheduler, large ecosystem
- **Prefect, Dagster:** more Pythonic, better local development
- **Kubeflow Pipelines, Vertex AI Pipelines, SageMaker Pipelines:** ML-native, cloud-integrated

Principles: steps are idempotent (rerunning is safe), outputs are cached by input hash, and every step logs what it consumed and produced.

4.4 Deep learning training essentials

- **Hardware:** GPUs for most DL; use mixed precision (bf16/fp16) to save memory and time
- **Distributed training:** DDP (replicate model, split data), FSDP and DeepSpeed ZeRO (shard model state across GPUs) for models too big for one GPU
- **Checkpointing:** save regularly so spot-instance interruptions do not lose days of work
- **Model formats:** PyTorch checkpoints, `safetensors` (safe, fast), ONNX (portable), GGUF (llama.cpp local inference), TensorRT (NVIDIA optimized)

4.5 For LLMs: prompt, RAG, or fine-tune?

This is the most important decision in LLM development. Go in this order and only escalate when the cheaper option fails:

1. **Prompting:** clear instructions, examples (few-shot), structured output. Cheapest, fastest to iterate
2. **RAG:** when the model lacks your private or fresh knowledge. Best for facts that change, citations, and permissioned data
3. **Fine-tuning:** when you need a consistent style, format or behavior, a smaller cheaper model matching a larger one on a narrow task, or lower latency. It is *not* a good way to add facts
4. **Pretraining or heavy continued pretraining:** almost never worth it for application teams

Fine-tuning techniques:

- **SFT (supervised fine-tuning):** learn from input-output examples

- **LoRA / QLoRA:** train small adapter matrices instead of all weights; cheap, fast, swappable per use case
- **Preference tuning (DPO, RLHF-style):** teach the model which of two answers is better
- **Distillation:** use a strong model's outputs to train a smaller one

Data quality dominates: a few hundred to a few thousand clean, diverse examples often beat tens of thousands of messy ones. Always hold out an eval set and compare against the untuned baseline with a good prompt, since beating nothing proves little.

Exercise 4: Track three experiments in MLflow or W&B on any small task, varying one thing each time (model, prompt, or hyperparameter). Write down which variable helped and by how much.

Module 5: LLM Application Patterns

5.1 Anatomy of an LLM call

- **Messages:** system instructions, user input, prior turns, tool results
- **Tokens:** the unit of cost and context. Input and output tokens are priced separately, output usually higher
- **Context window:** maximum tokens the model can see. More context costs more, adds latency, and long contexts can dilute attention; send what is needed, not everything
- **Sampling parameters:** temperature (randomness), top_p, max_tokens, stop sequences. Use low temperature for extraction and classification, higher for creative work
- **Streaming:** return tokens as they are generated; it dramatically improves perceived latency

5.2 Prompts are code

- Store prompts in files (YAML/Jinja/Markdown) in the repo, not buried in strings
- Version them, review them in pull requests, and tie every production request to a prompt version in logs
- Use templates with explicit variables; separate **instructions** from **untrusted data** (user text, retrieved documents) with clear delimiters

- Structure that works: role and goal, rules and constraints, output format, examples, then the input
- A **prompt registry** (Langfuse, PromptLayer, or your own table) lets you change a prompt without redeploying code, but changes still need evaluation

5.3 Structured outputs

Downstream code needs data, not prose.

- Ask for JSON that matches a schema; validate with Pydantic
- Use the provider's **structured output / tool calling** features, or constrained decoding libraries (Outlines, guidance) for self-hosted models
- **Always validate and retry:** if parsing fails, re-ask with the error message, up to a small limit, then fall back

5.3b Tool use (function calling)

The model does not run tools; it *requests* them. Your code executes and returns results.

1. You describe tools (name, description, JSON schema for arguments)
2. The model replies with a tool request
3. Your code validates arguments, runs the tool, returns the result
4. The model continues, possibly calling more tools, then answers

Tool design rules: clear names and descriptions, narrow single-purpose tools, validated inputs, safe by default (read-only first), and descriptive error messages the model can act on. **MCP (Model Context Protocol)** standardizes how tools and data sources are exposed to models so one tool server can serve many applications.

5.4 The RAG pipeline in detail

```
User query
  -> Query understanding (rewrite, decompose, classify, add history)
  -> Retrieval (vector + keyword, metadata filters, top-k 20-50)
  -> Reranking (cross-encoder, keep top 3-8)
  -> Context assembly (dedupe, order, trim to token budget, add source ids)
  -> Generation (instructions: answer only from context, cite sources)
  -> Post-processing (validate citations, guardrails, format)
  -> Response + logged trace
```

Improvements in rough order of payoff:

1. Fix parsing and chunking
2. Add hybrid search and a reranker
3. Query rewriting (turn 'what about its pricing?' into a standalone question)
4. Metadata filters (right tenant, right product, right date)
5. Parent-child retrieval and contextual chunks
6. Teach the model to say 'I do not know' when context is insufficient
7. Agentic retrieval: the model decides what to search for, and searches again if results are weak

Diagnose RAG by stage. If the answer is wrong, ask: was the right chunk in the index? Was it retrieved? Was it ranked high? Did the model use it? Each question points to a different fix.

5.5 Workflows vs agents

- **Workflow:** you define the steps in code (prompt chaining, routing, parallel calls, evaluator-optimizer loops). Predictable, cheaper, easier to test
- **Agent:** the model decides the next step in a loop (think, call tool, observe, repeat) until done. Flexible, but less predictable, slower and costlier

Start with the simplest thing: a single well-prompted call, then a workflow, and use an agent only when the path genuinely cannot be known in advance. Production agent essentials: step and token budgets, timeouts, loop detection, human approval for risky actions, persisted state so runs can resume, and full tracing of every step.

5.6 Memory and state

- **Short-term:** conversation history; trim or summarize when it grows
- **Long-term:** facts about users or tasks stored in a database or vector store and retrieved when relevant
- **Working state for agents:** a structured scratchpad or task state persisted between steps
- Keep servers **stateless**: store conversation state in a DB or cache keyed by session id, so any instance can serve any request

5.7 Caching

- **Exact-match cache:** same input, same output (Redis); safe for deterministic tasks

- **Semantic cache:** similar questions reuse answers; powerful but risks wrong hits, so use strict similarity thresholds
- **Prompt (prefix) caching:** providers cache long static prefixes (system prompt, documents), cutting cost and latency on repeated context
- Cache embeddings and retrieval results too

5.8 Routing and the LLM gateway

A gateway is a thin service between your apps and model providers. It centralizes:

- Provider abstraction (swap models without app changes)
- Routing (cheap model for easy tasks, strong model for hard ones)
- Fallbacks and retries across providers
- Auth, rate limits, quotas and per-team budgets
- Logging, cost tracking, guardrails and caching

Open options include LiteLLM, Portkey, Kong AI Gateway and cloud-native gateways; many teams also build their own, since this layer is core platform infrastructure.

Exercise 5: Build a small RAG service that answers questions over your 50 documents with citations and a structured JSON response. Then add a tool (for example, a calculator or a database lookup) and compare the workflow version against an agent version on the same 10 questions for cost, latency and accuracy.

Continued in Part B: evaluation, containers and APIs, serving and inference.

PART B: QUALITY AND PACKAGING

Module 6: Evaluation

6.1 Why evaluation is the center of everything

In classical software, tests tell you whether code is correct. In AI systems, **evals** tell you whether the system is *good enough*, and whether your last change made it better or worse. Without evals you are guessing, and every prompt tweak, model upgrade or chunking change becomes a gamble. Teams that move fast with AI are almost always teams with strong evals.

Offline evaluation runs before release on a fixed dataset. **Online evaluation** runs on live traffic after release. You need both.

6.2 Build an evaluation dataset

Start small and real:

- **Golden set:** 50-200 examples drawn from real or realistic user inputs, each with the expected outcome or a rubric describing what good looks like
- **Coverage:** include common cases, edge cases, ambiguous questions, out-of-scope questions (the right answer is a refusal), and adversarial inputs
- **Stratify** by category so you can see 'works for billing questions, fails for refunds'
- **Version** the dataset like code, and never tune on your test set. Keep a held-out slice you rarely look at
- **Synthetic data** (generated by an LLM from your documents) is a good way to bootstrap, but review a sample by hand and mix in real traffic as soon as you have it
- Add every production failure you find to the dataset, so the same bug cannot return silently

6.3 Kinds of metrics

Deterministic checks (cheap, run on everything):

- Output parses as valid JSON and matches the schema
- Required fields present, length limits respected, forbidden content absent
- Exact match or regex for classification and extraction
- Tool called correctly with valid arguments

Classical ML metrics: accuracy, precision, recall, F1, ROC-AUC for classifiers; RMSE, MAE for regression; BLEU/ROUGE only for narrow cases since they correlate poorly with quality for open-ended text.

Rubric and LLM-as-judge: use a strong model with a written rubric to grade outputs (correctness, completeness, tone, groundedness). Make it reliable:

- Grade one criterion at a time with a clear scale and definitions
- Ask for reasoning before the score, use low temperature
- Prefer pairwise comparison ('which is better, A or B') for subtle quality differences, and randomize order to defeat **position bias**
- Watch for **verbosity bias** (longer looks better) and **self-preference** (a model favors its own style)
- **Calibrate against humans:** label 50-100 examples yourself, check that the judge agrees with you, and refine the rubric until it does

Human review: the gold standard. Use it to calibrate judges, review samples regularly, and decide hard cases.

6.4 Evaluating RAG

Evaluate retrieval and generation *separately*, then end to end.

Stage	Question	Metrics
Retrieval	Did we fetch the right chunks?	Recall@k, precision@k, MRR, nDCG, hit rate
Context	Is the context relevant and sufficient?	Context relevance, context recall
Generation	Is the answer supported by the context?	Faithfulness / groundedness, citation accuracy
Answer	Does it answer the question well?	Answer relevance, correctness vs reference
System	Did the user succeed?	Task success, deflection rate, escalation rate

Build a retrieval test set as (question, ids of relevant chunks) pairs. This single asset lets you tune chunking, embeddings, hybrid search and rerankers quickly and cheaply without calling a generator at all.

Tools: Ragas, DeepEval, promptfoo, TruLens, Braintrust, LangSmith, Langfuse, Arize Phoenix, Inspect.

6.5 Evaluating agents

Agents need more than final-answer grading:

- **Task success** in a controlled environment (did the booking get made, did the tests pass)
- **Trajectory quality:** right tools, right order, no loops, no unnecessary steps
- **Efficiency:** steps, tokens, cost and wall-clock time per task
- **Safety:** did it ever attempt a forbidden or destructive action
- **Variance:** run each task several times, since agents are stochastic; report pass rates, not single runs

6.6 Safety and red-team evaluation

Test prompt injection, jailbreaks, data leakage, toxic output, bias and refusal accuracy (neither over-refusing nor under-refusing). Keep these as a standing suite and add automated adversarial generation as you mature.

6.7 Evals in CI/CD

- On every pull request: fast deterministic checks plus a small stratified eval subset
- Nightly or pre-release: the full suite including expensive judge-based and agent evals
- **Quality gate:** block merges that drop key metrics below a threshold or regress more than a set margin versus the main branch
- Because outputs are noisy, run enough samples to tell a real change from randomness, and look at per-category results, not just the average
- Store results with the prompt version, model version, dataset version and commit, so you can compare any two runs

6.8 Online evaluation and the error-analysis loop

- **A/B tests and shadow traffic:** compare a candidate against production on live inputs
- **User signals:** thumbs up/down, edits, copy events, retries, escalations to humans
- **Sampled production grading:** run your judge on a random 1-5% of real traffic to track quality over time
- **Error analysis:** read at least 50 real failures, label them into categories (retrieval miss, wrong tool, format break, hallucination, unclear question), count them, and fix the biggest bucket first. This habit is worth more than any framework

Exercise 6: Write a 50-example golden set for your RAG service from Module 5, add three deterministic checks and one calibrated LLM judge for groundedness, then change your chunk size and see whether the numbers move. Wire it into a script that exits non-zero if quality drops.

Module 7: Packaging, Containers and APIs

7.1 Why containers

A container bundles your code, dependencies and runtime into one immutable image. The same image runs on your laptop, in CI, in staging and in production, which kills 'works on my machine'. It is also the unit that Kubernetes, Cloud Run, ECS and every serverless container platform deploys.

7.2 Writing a good Dockerfile

- Use a small, pinned base image (`python:3.12-slim` , not `latest`)
- **Multi-stage builds:** build and install dependencies in one stage, copy only what you need into the final image
- Order layers for caching: copy the lockfile and install dependencies first, then copy source code, so code edits do not reinstall everything
- Run as a **non-root** user; do not bake secrets into the image
- Add a `.dockerignore` (no `.git` , notebooks, datasets, `.env`)
- Define a `HEALTHCHECK` or expose health endpoints
- Keep model weights out of the image when large; load from object storage or a mounted volume at start-up, or use a dedicated model image strategy for fast scaling
- GPU images: start from an official CUDA runtime image matching your framework version
- Scan images for vulnerabilities (Trivy, Grype) in CI, and tag images with the git SHA, never only `latest`

7.3 Designing the API

Most AI apps expose an HTTP API. FastAPI is the common Python choice.

- **Typed contracts:** Pydantic request and response models give validation and auto-generated docs
- **Versioning:** `/v1/...` so you can evolve without breaking clients
- **Consistent errors:** one error shape with a code, message and request id
- **Request ids** on every call, propagated to logs and traces
- **Idempotency keys** for operations with side effects, so retries are safe

- **Authentication:** API keys for service clients, OAuth/JWT for end users; authorization checks per resource and per tenant
- **Rate limiting and quotas** per user or tenant, to protect you from both abuse and runaway cost

7.4 Streaming responses

LLM calls take seconds, so stream. Use **Server-Sent Events (SSE)** for simple one-way token streaming, or WebSockets when you need two-way interaction. A minimal sketch:

```
from fastapi import FastAPI
from fastapi.responses import StreamingResponse

app = FastAPI()

@app.post('/v1/chat')
async def chat(req: ChatRequest):
    async def event_stream():
        async for token in llm.stream(req.messages):
            yield f'data: {token}\n\n'
        yield 'data: [DONE]\n\n'
    return StreamingResponse(event_stream(), media_type='text/event-stream')
```

Remember that proxies and load balancers must be configured not to buffer streamed responses, and that clients can disconnect mid-stream, so cancel the upstream model call when they do (it saves money).

7.5 Sync, async and background work

- **Synchronous request/response:** quick tasks, under a few seconds
- **Async I/O (asyncio):** essential for LLM apps, since almost all time is spent waiting on network calls. One process can serve many concurrent requests
- **Background jobs with a queue:** long tasks (document processing, long agent runs, batch embeddings) return a job id immediately; workers process from a queue (Celery, RQ, Dramatiq, SQS, Cloud Tasks, Temporal for durable workflows); clients poll or receive a webhook

Rule of thumb: if it can take longer than about 30 seconds, or must survive a restart, make it a job.

7.6 Making services production-friendly

- **Stateless:** no user state in process memory; use a DB or Redis
- **Health endpoints:** *liveness* (process is alive, restart if not) and *readiness* (can serve traffic: dependencies reachable, model loaded)
- **Graceful shutdown:** finish in-flight requests on SIGTERM before exiting
- **Timeouts everywhere:** every outbound call has a timeout; never wait forever
- **Connection pooling** for databases and HTTP clients
- **Structured JSON logging** with request id, from the very first day
- **Concurrency settings:** uvicorn/gunicorn workers sized to CPU, async handlers for I/O; blocking CPU-heavy work (embedding on CPU) goes to a thread pool or a separate service

Exercise 7: Containerize your RAG service with a multi-stage Dockerfile, add `/healthz` and `/readyz`, a streaming `/v1/chat` endpoint with a request id in every log line, and run it with docker-compose together with its vector DB and Redis.

Module 8: Model Serving and Inference

8.1 Hosted API or self-hosted model?

Factor	Hosted API (Anthropic, OpenAI, Google, etc.)	Self-hosted open model
Time to start	Minutes	Days to weeks
Quality	Frontier models	Strong but usually a step behind at the top end
Cost at low volume	Cheap, pay per token	Expensive (idle GPUs)
Cost at very high, steady volume	Can get large	Often cheaper per token
Data control	Governed by provider agreement	Full control, data never leaves
Ops burden	Low	High (GPUs, scaling, upgrades)

Factor	Hosted API (Anthropic, OpenAI, Google, etc.)	Self-hosted open model
Customization	Prompting, some fine-tuning	Full: fine-tuning, quantization, custom kernels
Latency control	Limited	Fully tunable

Default to hosted APIs behind your own gateway. Move to self-hosting when you have strict data residency needs, very high steady volume, a narrow task a small model can handle, or special latency requirements. Many mature systems are hybrid: a frontier API for hard tasks, small self-hosted models for embeddings, reranking, classification and cheap routing.

8.2 Inference modes

- **Online (real-time):** request in, response out, latency-sensitive
- **Batch (offline):** score millions of records on a schedule; maximize throughput, ignore latency; far cheaper per item (many providers offer discounted batch APIs)
- **Streaming:** continuous event processing (fraud scoring on a Kafka stream)

8.3 Serving classical and DL models

Wrap the model in a service with a clear input/output contract, and separate **model loading** (once at start-up) from **prediction** (per request). Tools: BentoML, Ray Serve, TorchServe, NVIDIA Triton Inference Server, KServe on Kubernetes, and managed options such as SageMaker, Vertex AI and Azure ML endpoints.

Performance techniques: **dynamic batching** (group requests that arrive within a few milliseconds), model optimization (ONNX Runtime, TensorRT), lower precision, and running CPU models with enough threads and no accidental thread oversubscription.

8.4 How LLM inference actually works

Understanding this explains almost every cost and latency tradeoff.

- **Prefill:** the model processes the whole input prompt in parallel. This is compute-bound and determines **time to first token (TTFT)**
- **Decode:** the model generates output one token at a time. This is memory-bandwidth-bound and determines **inter-token latency** and streaming speed

- **KV cache:** to avoid recomputing attention over earlier tokens, the model stores keys and values for every token in memory. It grows with sequence length and with the number of concurrent requests, and is often what limits concurrency, not the weights
- **Throughput vs latency:** batching many requests together makes the GPU efficient (high throughput) but each request may wait longer. Serving is always a tradeoff on this curve

Key optimizations used by modern engines:

- **Continuous batching:** new requests join a running batch at each step instead of waiting for the whole batch to finish
- **PagedAttention** (vLLM): manages the KV cache in small blocks like virtual memory, cutting waste and raising concurrency
- **Prefix caching:** reuse KV cache for shared prompt prefixes such as a long system prompt
- **Speculative decoding:** a small draft model proposes several tokens that the large model verifies in one pass, speeding up generation without changing output quality
- **Tensor / pipeline parallelism:** split a model across several GPUs when it does not fit on one
- **FlashAttention and fused kernels:** faster, more memory-efficient attention

8.5 Quantization

Quantization stores weights (and sometimes activations or the KV cache) in fewer bits to cut memory and speed up inference.

- **bf16/fp16:** the standard full-quality baseline (2 bytes per parameter)
- **fp8 / int8:** about half the memory with usually negligible quality loss
- **int4 (GPTQ, AWQ, GGUF Q4 variants):** about a quarter of the memory, noticeable but often acceptable quality loss; great for local and cost-sensitive serving

Always evaluate the quantized model on *your* eval set, since the loss varies by task.

8.6 Sizing GPUs

Rough rule: **weights memory = parameters x bytes per parameter.** An 8B-parameter model at fp16 is about 16 GB; at int4 about 4-5 GB. Add memory for the KV cache (grows with context length x concurrent requests), activations and overhead. A model that barely fits in memory leaves no room for concurrency, so leave headroom. Choose GPUs by memory first, then memory bandwidth (for decode speed), then compute.

8.6b Inference engines

- **vLLM:** the popular open-source default; high throughput, OpenAI-compatible API
- **SGLang:** strong at structured generation and prefix reuse
- **TensorRT-LLM:** maximum performance on NVIDIA hardware, more setup
- **Hugging Face TGI:** mature, easy to deploy
- **llama.cpp / Ollama:** CPU and consumer-GPU inference, ideal for local development and edge
- **Managed endpoints:** Bedrock, Vertex AI, Together, Fireworks, Modal, Baseten, and similar, when you want open models without running GPUs

8.7 Scaling GPU services

- Autoscale on **queue depth, concurrent requests or tokens per second**, not CPU utilization
- **Cold starts** are painful: loading tens of GB of weights takes minutes. Mitigate with fast storage, cached weights on nodes, warm pools and a minimum replica count
- Use **spot/preemptible instances** for batch and tolerant workloads, on-demand for latency-critical ones
- **Multi-LoRA serving:** one base model serves many fine-tuned adapters, swapping them per request, which is far cheaper than a full model per customer
- Serve **embedding and reranker models** separately from generation models, since their scaling profiles differ

Exercise 8: Run a small open model (for example a 3B-8B instruct model) with vLLM or Ollama, call it through its OpenAI-compatible endpoint, then load-test with 1, 8 and 32 concurrent requests. Record TTFT, tokens per second and throughput at each level, and plot the tradeoff.

PART C: OPERATIONS

Module 9: CI/CD and Infrastructure

9.1 A CI/CD pipeline for AI applications

Continuous Integration (on every pull request):

1. Lint, format check, type check

2. Unit tests (parsers, chunkers, prompt builders, API logic)
3. Secret scanning and dependency vulnerability scan
4. Build the container image
5. Integration tests against ephemeral services (DB, vector store)
6. **Eval gate:** fast eval subset; block on regressions (Module 6)

Continuous Delivery (on merge to main):

1. Build and tag the image with the commit SHA; push to a registry
2. Deploy automatically to **staging**
3. Run smoke tests and the full eval suite against staging
4. Promote to **production** automatically or with a manual approval, using a safe rollout strategy (9.4)
5. Watch dashboards and alerts during rollout; auto-rollback on failure

Tools: GitHub Actions, GitLab CI, Jenkins, CircleCI, Buildkite; for deployments, ArgoCD or Flux (GitOps), or the cloud provider's own pipelines.

9.2 Environments

- **Local:** docker-compose, mocked or cheap models
- **Dev:** shared sandbox, small data, low quotas
- **Staging:** production-like infrastructure and config, realistic data (sanitized), used for final verification
- **Production:** locked down, audited, alerting on

Keep config differences small and explicit. Never let staging use production data without the same access controls.

9.3 Infrastructure as Code and runtime platforms

IaC (Terraform, OpenTofu, Pulumi, CloudFormation, CDK) defines infrastructure in versioned code, so environments are reproducible, reviewable and recoverable.

Where your containers run, from simplest to most flexible:

- **Serverless containers** (Cloud Run, AWS App Runner, Azure Container Apps, Modal): scale to zero, minimal ops. Great starting point for API services
- **Managed container services** (ECS/Fargate): more control, still low ops

- **Kubernetes** (EKS, GKE, AKS): the standard for complex, multi-service and GPU workloads
- **ML platforms** (SageMaker, Vertex AI, Azure ML, Databricks): managed training and serving

Kubernetes vocabulary you should know: **Pod** (one or more containers), **Deployment** (desired replicas, rolling updates), **Service** (stable network address), **Ingress / Gateway** (external routing), **HPA** (autoscaling), **ConfigMap / Secret** (config and secrets), **resource requests and limits** (CPU, memory, GPU), **node pools** (GPU vs CPU nodes), **namespaces** (isolation). Do not adopt Kubernetes before you need it; a serverless container platform is often enough for a long time.

9.4 Safe deployment strategies

Strategy	How it works	Use when
Rolling	Replace instances gradually	Default for stateless services
Blue-green	Run new version beside old, switch traffic at once, keep old for instant rollback	You need fast, clean rollback
Canary	Send 1%, 5%, 25%, 100% of traffic to the new version while watching metrics	Risky changes; most AI releases
Shadow	Copy live traffic to the new version, discard its responses, compare offline	Testing a new model or pipeline with zero user risk
A/B test	Split users and measure business outcomes	Choosing between two designs

Use **feature flags** (LaunchDarkly, Unleash, Statsig, or a simple config table) to separate *deploying* code from *releasing* behavior. They let you roll out a new prompt or model to 5% of users, and switch it off in seconds without a redeploy.

9.5 What gets versioned and released in AI systems

A release is more than a code deploy. Each of these can change behavior and must be versioned, evaluated and rollback-able:

- Application code and container image

- **Prompts** and prompt templates
- **Model and model version** (pin exact provider model ids; do not follow a moving alias in production unless you accept silent changes)
- **Retrieval index and embedding model.** Rebuild into a *new* index, evaluate it, then switch an alias to it (blue-green for indexes), keeping the old one for rollback
- Tool definitions and agent configuration
- Guardrail rules and thresholds

When a provider deprecates a model, treat the migration as a normal release: run your evals on the new model first.

9.6 Continuous training (for ML models you train)

Automate retraining triggered by a schedule, enough new data, or detected drift. The pipeline trains, evaluates against the current production model on a fixed test set plus recent data, and **only registers or promotes the new model if it passes gates**. A human approval step is sensible for high-risk models.

9.7 Release checklist

- Evals pass and no category regressed beyond tolerance
- Load test done at expected peak with headroom
- Dashboards, alerts and runbook updated
- Rollback path tested, not just documented
- Cost per request estimated and within budget
- Security review for new tools, data access or permissions
- Feature flag ready as a kill switch

Exercise 9: Create a GitHub Actions pipeline for your RAG service: lint, test, build and push an image tagged with the commit SHA, run your eval script as a gate, and deploy to a Cloud Run or equivalent staging service. Then write the Terraform (or your IaC of choice) for the service and its secrets.

Continued: observability, guardrails and security, cost and latency, reliability.

Module 10: Observability and Monitoring

10.1 Why AI systems need more than normal monitoring

A normal service is 'up' or 'down'. An AI service can be up, fast, returning HTTP 200, and *quietly wrong*. Observability for AI therefore adds **quality** to the usual health signals. You need to answer three questions at any time: Is it working? Is it good? What is it costing?

10.2 The pillars

- **Logs:** discrete events, structured as JSON, searchable by request id
- **Metrics:** numeric time series (latency, error rate, tokens, cost), cheap to store and great for alerting
- **Traces:** the full path of one request across services and steps, the single most useful tool for debugging RAG and agents
- **Evaluations and feedback (the AI-specific pillar):** quality scores and user signals attached to traces

10.3 What to capture for every LLM request

- Trace id, request id, user/tenant id (pseudonymized), timestamp, environment
- Prompt template version, model name and exact version, sampling parameters
- Input and output tokens, **time to first token**, total latency, estimated cost
- For RAG: query, rewritten query, retrieved document ids and scores, reranker scores, final context ids
- For agents: every step, tool name, tool arguments, tool result summary, step count
- Outcome: success, error type, retries, fallback used, guardrail triggers
- Feedback: thumbs, edits, escalations, follow-up rephrasing

Privacy: decide what you may store. Redact PII and secrets before logging, apply retention limits, restrict who can view raw prompts, and consider logging hashes or metadata only for sensitive tenants.

10.4 Tracing

Use **OpenTelemetry** as the standard instrumentation layer, with spans for each stage: `api_request` > `retrieve` > `rerank` > `llm_call` > `tool_call` > `postprocess`. A trace view answers 'why was this answer bad' in a minute: wrong chunks retrieved? tool error?

the prompt was truncated? Tools built for this: Langfuse, LangSmith, Arize Phoenix, Helicone, Braintrust, W&B Weave, Datadog LLM Observability, and any OTel-compatible backend (Jaeger, Grafana Tempo).

10.5 Metrics and dashboards

Golden signals (every service): latency (p50, p95, p99), traffic (requests per second), errors (rate and type), saturation (CPU, memory, GPU, queue depth).

AI-specific metrics:

- Tokens per request, cost per request and per user or tenant, daily spend
- TTFT and inter-token latency for streaming
- Cache hit rate (exact, semantic, prompt cache)
- Format and schema failure rate, retry rate, fallback rate
- Refusal rate, guardrail trigger rate
- Retrieval metrics: empty-result rate, average top score
- Quality: sampled judge scores (groundedness, relevance), thumbs up ratio, escalation rate
- Self-hosted: GPU utilization and memory, KV cache usage, requests waiting, tokens per second

Build one dashboard per audience: an on-call health view, a quality view for the AI team, and a cost view for the business.

10.6 Drift: why systems decay

- **Data drift:** input distribution changes (new products, new slang, new languages)
- **Concept drift:** the right answer changes for the same input (policies, prices, user behavior)
- **Knowledge drift:** the index goes stale or documents change
- **Provider drift:** a hosted model is updated and behaves differently; pinned versions reduce but do not eliminate this
- **Upstream drift:** a source schema, parser or API changes

Detection: for tabular features, statistical tests (PSI, KS test, chi-square) comparing live to training distributions; for text, track embedding distribution shift and topic mix of incoming queries; for outputs, track quality scores over time. For LLM apps the most

practical drift detector is **running your evals continuously on sampled production traffic** and a fixed canary set daily.

10.7 Alerting

- Define **SLOs** first (for example, 99.5% of requests succeed, p95 time to first token under 2 seconds, groundedness score above 0.9 on sampled traffic)
- Alert on SLO burn and symptoms users feel, not on every metric wobble
- Every alert links to a **runbook**: what it means, how to check, how to mitigate
- Page humans only for urgent, actionable problems; send the rest to tickets or channels, to avoid alert fatigue
- Add budget alerts for cost spikes, which are an outage of a different kind

10.8 Closing the loop

Observability only pays off if it feeds improvement. Route low-scoring and thumbs-down traces into a review queue, label them, add them to the eval dataset, fix the root cause, and verify the fix with your CI evals. This is the engine of Level 3 maturity from Module 1.

Exercise 10: Add OpenTelemetry or Langfuse tracing to your RAG service with spans for retrieval, rerank and generation. Build a dashboard showing p95 latency, tokens, cost per request and thumbs-down rate, and set one alert with a runbook.

Module 11: Guardrails, Security and Safety

11.1 The threat model for AI applications

AI apps have the usual web security surface plus new ones: the model follows instructions found in *data*, it can be talked into things, and it can be wired to tools that act in the real world. Start every design with: what can go wrong, who could cause it, and what is the worst outcome.

11.2 Main risks (aligned with the OWASP Top 10 for LLM Applications)

- **Prompt injection:** instructions hidden in user input (direct) or in retrieved documents, web pages, emails or tool outputs (**indirect**) hijack the model. The most important LLM-specific risk

- **Sensitive information disclosure:** the model reveals system prompts, other users' data, secrets or PII
- **Insecure output handling:** model output passed unsanitized into SQL, shell, HTML or code execution
- **Excessive agency:** the model has more tools, permissions or autonomy than the task needs
- **Data and model poisoning:** malicious content in training data, fine-tuning sets or the knowledge base
- **Supply chain:** compromised packages, model files or plugins (prefer `safetensors`, scan dependencies, pin versions)
- **Unbounded consumption:** denial-of-wallet through huge inputs, loops or abuse
- **Hallucination and overreliance:** confident wrong answers that users trust

11.3 Defense in depth

No single control stops prompt injection, so layer defenses and assume the model *can* be fooled.

1. **Least privilege:** give tools and agents only the minimum permissions; read-only by default; scoped, short-lived credentials; separate credentials per tenant
2. **Treat all model output as untrusted input:** validate against schemas, escape before rendering, never execute generated code or SQL without sandboxing and allow-lists
3. **Separate instructions from data:** delimit untrusted content clearly and tell the model it is data, not commands (helpful, not sufficient)
4. **Human-in-the-loop for high-impact actions:** payments, deletions, external emails, production changes require confirmation
5. **Authorization at the data layer:** enforce user permissions in retrieval and tool calls (metadata filters, row-level security), never rely on the prompt to keep secrets
6. **Limit the lethal combination:** an agent that can read private data, see untrusted content *and* send data outward is an exfiltration risk; break at least one leg of that triangle
7. **Sandbox code execution** in isolated containers with no network and strict resource limits
8. **Rate limits, budgets and quotas** per user, with step and token caps for agents

11.4 Input and output guardrails

- **Input checks:** length limits, language detection, PII detection and masking, injection and jailbreak classifiers, topic restrictions, malware/URL checks for uploaded content
- **Output checks:** schema validation, toxicity and policy classifiers, PII and secret leak detection, groundedness checks against retrieved context, citation verification, brand and legal rules
- **Actions on failure:** block with a safe message, retry with a corrective instruction, redact, escalate to a human, or fall back to a safer path
- **Tools:** provider moderation and safety classifiers, Llama Guard-style models, NVIDIA NeMo Guardrails, Guardrails AI, Presidio for PII, Lakera, plus your own rules. Place them in the **gateway** so every application gets them

Every guardrail has false positives and adds latency. Measure both, run cheap checks first and heavy ones only when needed, and sample-review blocked traffic.

11.5 Privacy, compliance and governance

- Know where data flows: which provider, which region, what retention, whether it is used for training (check agreements, use zero-retention options when required)
- Minimize what you send to models; mask PII where possible
- Encrypt data in transit and at rest; manage keys properly
- Keep **audit logs** of who accessed what, and of model and prompt changes
- Map to relevant regulations and standards (GDPR, India's DPDP Act, HIPAA, SOC 2, ISO 27001, the EU AI Act, NIST AI RMF) according to your domain and users
- Document each AI system: purpose, data, limitations, evaluation results, owners (model cards, system cards)
- Plan for incidents: detection, kill switch, communication, post-mortem

11.6 Classical security hygiene still applies

AuthN/AuthZ, TLS, secrets management and rotation, dependency scanning, container hardening, network policies, WAF, logging of security events, and regular penetration testing including AI-specific red teaming.

Exercise 11: Write a threat model for your RAG service in one page: assets, entry points, top five threats, and a control for each. Then write five injection test cases (including one hidden inside a retrieved document), run them, and add them to your standing eval suite.

Module 12: Cost and Latency Optimization

12.1 Know where the money and time go

For hosted LLM apps, cost is roughly **(input tokens x input price) + (output tokens x output price)** summed over every call in a request. A single user query in a RAG or agent system can trigger many calls, so measure *per request* and *per successful task*, not per call. Latency similarly adds up across retrieval, reranking, several model calls and tools.

First instrument, then optimize the biggest line item. Guessing wastes time.

12.2 Cost levers, roughly ordered by payoff

1. **Right-size the model:** use the smallest model that passes your evals for each step. Route easy tasks to cheap models and escalate hard ones. This is usually the largest saving
2. **Cut tokens:** shorter system prompts, fewer and better retrieved chunks (rerank, then keep the top few), summarize long history, strip boilerplate, ask for concise output and cap `max_tokens`
3. **Prompt caching:** place static content (instructions, tool definitions, long documents) at the *start* of the prompt so provider caches reuse it at a discount
4. **Response and semantic caching** for repeated questions
5. **Batch APIs** for non-urgent work, often priced at a substantial discount
6. **Avoid unnecessary calls:** deterministic code for deterministic steps, skip the LLM when a rule or classifier suffices, stop agent loops early
7. **Self-host small models** for high-volume narrow tasks (embeddings, reranking, classification), as in Module 8
8. **Fine-tune a small model** to replace a large one on a stable narrow task, when evals show parity
9. **Budgets and quotas** per tenant, with alerts, to prevent runaway spend

12.3 Latency levers

- **Stream** output so users see text immediately; perceived latency is what matters
- **Parallelize** independent steps (retrieve from several sources at once, run guardrails alongside generation)
- **Reduce prompt size** (prefill time scales with input) and output length (decode time scales with output)

- Use **smaller or faster models** for latency-critical steps; reserve large models for the final answer
- **Cache** aggressively at every layer (embeddings, retrieval, prompts)
- **Speculative and prefetch work:** start retrieval while the user is still typing, or precompute common answers
- Co-locate services and keep connections warm; choose regions near users and near the model endpoint
- Self-hosted: continuous batching, prefix caching, quantization, speculative decoding (Module 8)
- Set **timeouts and hedged requests** to cut tail latency (p99), because users feel the slow ones

12.4 The quality-cost-latency triangle

You cannot maximize all three. Define the product requirement explicitly (for example 'p95 under 3 seconds, under 2 cents per query, at least 90% task success') and use your eval suite to find the cheapest configuration that meets it. Every optimization above must be re-evaluated for quality, since a cheaper path that quietly hurts accuracy is not a saving.

12.5 A worked estimate

Suppose a support bot answers 100,000 queries a month. Each query uses about 3,000 input tokens (instructions plus retrieved context) and 400 output tokens. That is 300 million input tokens and 40 million output tokens a month. Multiply by your model's per-million-token prices to get the monthly bill, then ask: what if routing sends 70% of queries to a model ten times cheaper? What if prompt caching covers 1,500 of those input tokens at a discount? Running these what-ifs on paper before building is a core system design skill.

Exercise 12: Instrument token and cost per request in your service. Produce a table of where cost goes (retrieval, rerank, generation) and implement two optimizations, such as a cheaper-model router and prompt caching. Prove with your evals that quality did not drop.

Module 13: Reliability and Resilience

13.1 Assume everything fails

Model APIs rate-limit, time out and have outages. Vector DBs slow down. Tools return garbage. Reliable AI systems are built from the assumption that each dependency will fail sometimes, and that the system must degrade gracefully instead of collapsing.

13.2 Core patterns

- **Timeouts:** every outbound call has one; no unbounded waits
- **Retries with exponential backoff and jitter** for transient errors (429, 5xx, network); never retry blindly on non-idempotent actions; cap attempts
- **Rate-limit handling:** respect `retry-after` headers, use client-side token buckets and queues to stay under quotas
- **Circuit breakers:** after repeated failures, stop calling the failing dependency for a cool-down period and fail fast
- **Fallbacks:** a secondary model or provider, a cached answer, a simpler non-LLM path, or an honest 'I cannot answer right now'
- **Bulkheads:** isolate resources so one failing feature or tenant cannot starve everything else
- **Load shedding and backpressure:** reject or queue excess traffic deliberately rather than falling over
- **Idempotency:** safe to repeat, especially for agent actions and queue consumers
- **Dead-letter queues** for jobs that keep failing, so they can be inspected and replayed
- **Durable execution** (Temporal, Step Functions, LangGraph persistence) for long agent workflows that must survive crashes and resume

13.3 Graceful degradation ladder

Design the order in which the system gets simpler under stress: full pipeline, then skip the reranker, then use a smaller model, then serve cached or canned answers, then show a clear error with next steps. Decide this on purpose rather than discovering it during an outage.

13.4 Multi-provider resilience

Abstract providers behind your gateway, keep prompts tested on at least two models, and fail over automatically. Be aware that prompts rarely transfer perfectly, so the fallback model needs its own evals, and quality on fallback should be monitored.

13.5 Reliability targets and practice

- Set **SLIs and SLOs** (availability, latency, quality) and an **error budget**: when you burn it, slow feature releases and fix reliability
- **Load test** before launch and after major changes (Locust, k6) at realistic prompt sizes and concurrency, since token counts change everything
- **Chaos and failure drills:** simulate provider outages and slow dependencies in staging
- **Disaster recovery:** backups of databases, vector indexes and configs; tested restore; documented RTO and RPO
- **Incident process:** on-call rota, severity levels, runbooks, blameless post-mortems that produce concrete fixes and new tests or evals

Exercise 13: Add timeouts, retries with backoff, a circuit breaker and a provider fallback to your gateway client. Then simulate a provider outage (return errors from a mock) and confirm that users still get a graceful response and that your dashboard shows the fallback rate.

Continued in Part D: system design, governance, capstones and roadmap.

PART D: SYSTEM DESIGN AND PRACTICE

Module 14: System Design for AI Applications

14.1 A framework for any AI system design

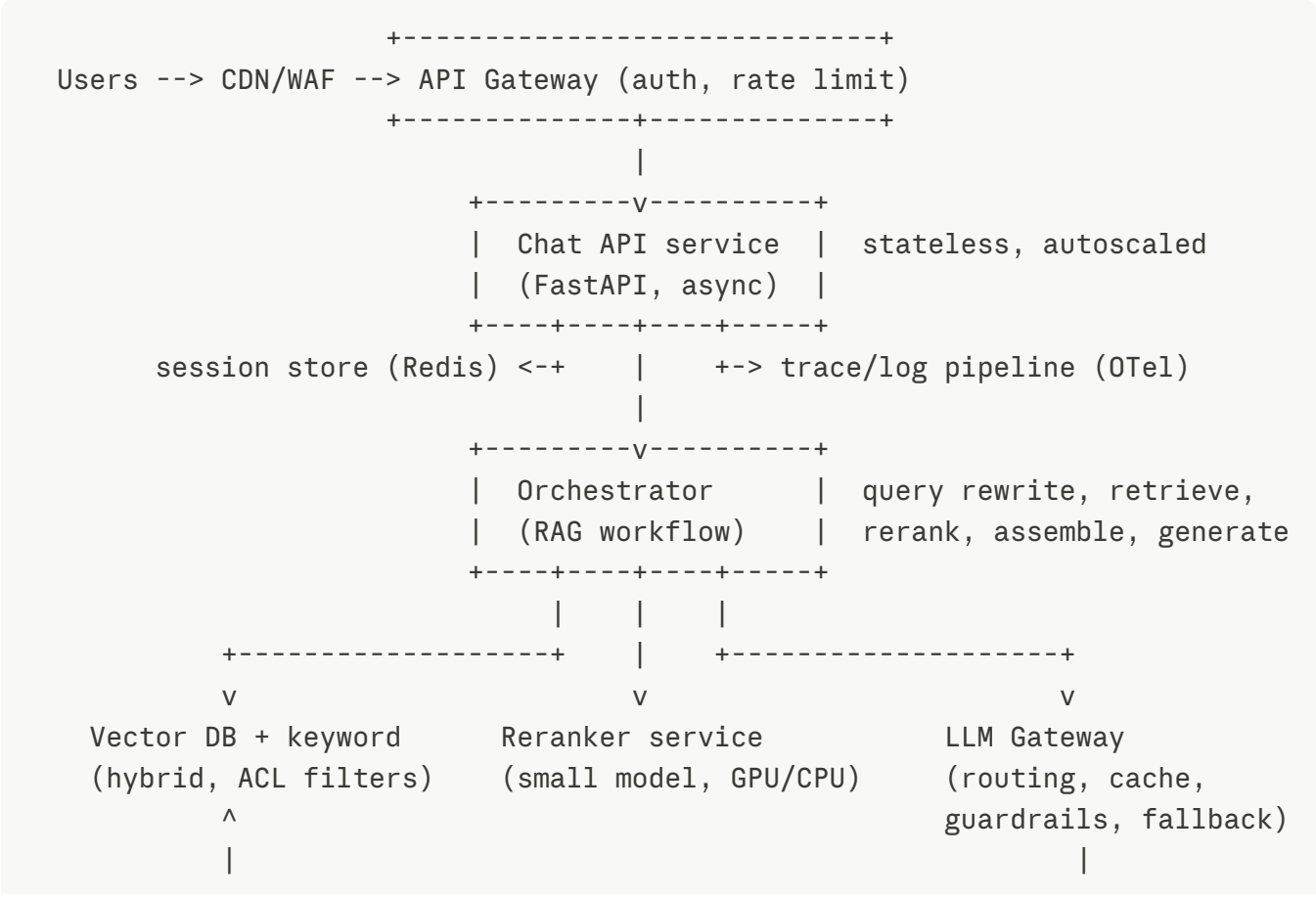
When you design (or are asked to design) an AI-powered system, walk through these steps in order. They keep you from jumping to tools before understanding the problem.

1. **Clarify the problem and users.** Who uses it, for what task, how often, what does success look like, what is the cost of a wrong answer?
2. **Define requirements.** *Functional* (what it does) and *non-functional*: latency, throughput, availability, accuracy or quality targets, cost ceiling, data residency, compliance

3. **Estimate scale.** Requests per second at peak, tokens per request, data volume, growth. Do the arithmetic (see 12.5)
4. **Choose the AI approach.** Rules? Classical ML? Prompting, RAG, fine-tuning, agent? Start with the simplest that could work
5. **Sketch the data flow.** Sources, ingestion, storage, retrieval, model calls, outputs, feedback
6. **Design the components.** API, orchestration, model layer, data stores, queues, cache, gateway
7. **Plan evaluation.** Datasets, metrics, gates, online measurement
8. **Plan deployment and operations.** Environments, CI/CD, rollout, observability, alerting
9. **Address cross-cutting concerns.** Security, privacy, cost, reliability, governance
10. **Identify risks and tradeoffs.** Say what you chose, what you rejected, and why. Plan the iteration roadmap

The single most valuable habit: **state tradeoffs explicitly.** Good design is not the 'right' architecture; it is a defensible one for the stated constraints.

14.2 Reference architecture: enterprise RAG assistant



```
 Ingestion pipeline                                  Hosted APIs / vLLM
 (parse, chunk, embed,
  incremental sync) <-- Source systems (wiki, docs, tickets)

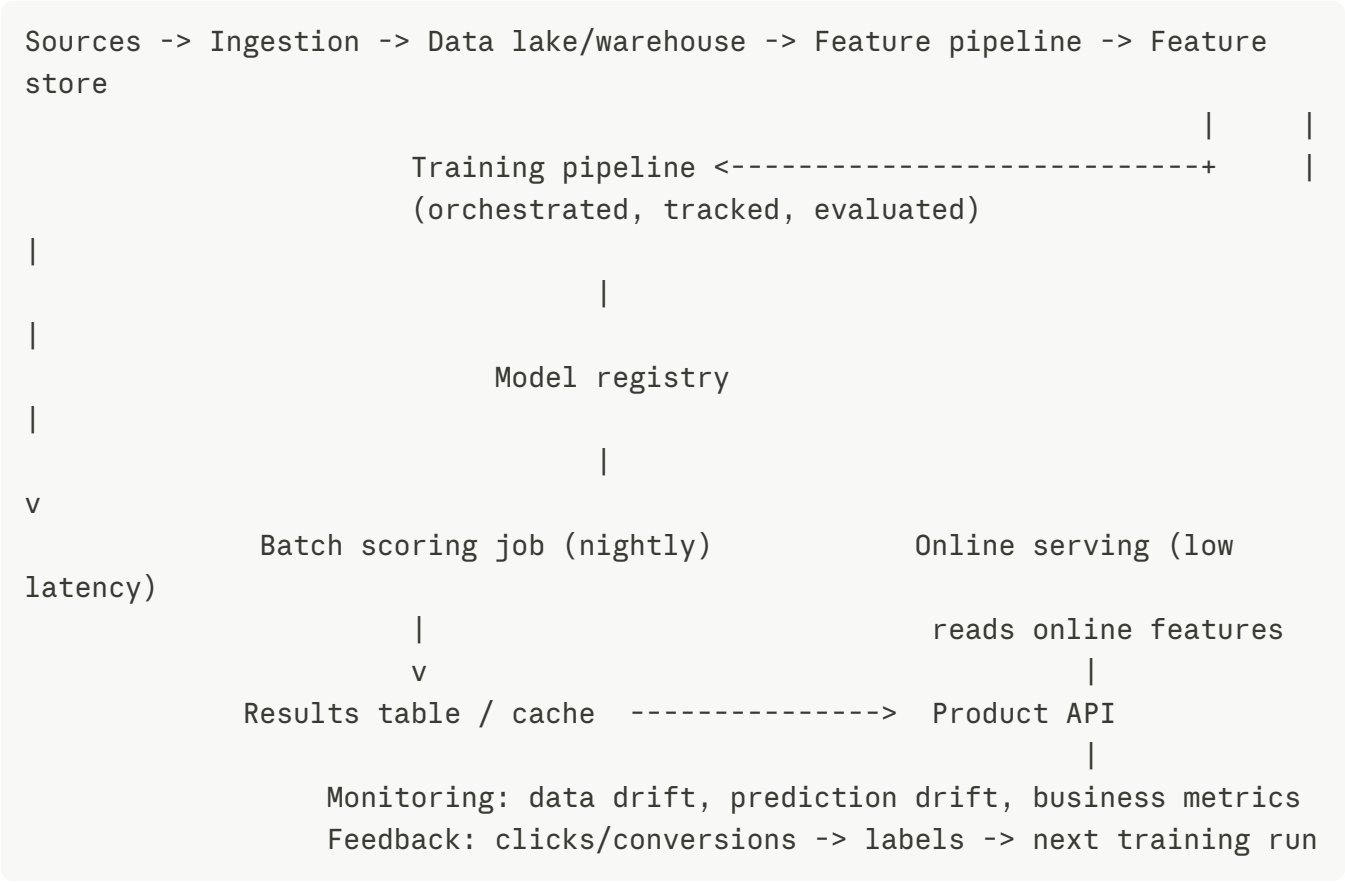
Cross-cutting: secrets manager, IaC, CI/CD + eval gate, dashboards,
alerting, feedback store, audit log, cost tracking
```

Walkthrough of one request: the user is authenticated at the gateway and rate limited; the chat service loads session history from Redis; the orchestrator rewrites the follow-up into a standalone question; hybrid retrieval runs with the user's access filters; a reranker picks the best chunks; the context is assembled within a token budget with source ids; the LLM gateway applies input guardrails, calls the model (with prompt caching and a fallback provider), and streams tokens back; output guardrails verify citations; the full trace, tokens, cost and any later feedback are stored; sampled traces are graded by an online judge.

Key design decisions to be able to justify: hybrid vs pure vector search; chunking strategy; where permissions are enforced; hosted vs self-hosted models; sync streaming vs async jobs; how the index is rebuilt and swapped; what happens when the model provider is down.

14.3 Reference architecture: batch ML scoring and recommendations

This is the classical ML pattern, still extremely common.



Important ideas: the same feature logic for training and serving (feature store) to avoid skew; offline scoring where freshness allows (far cheaper); a two-stage design for recommendations (candidate retrieval, then ranking); delayed labels handled explicitly in monitoring.

14.4 Reference architecture: agent platform

Components: an **agent runtime** (loop, state machine or graph) with persisted state; a **tool layer** (internal APIs, MCP servers) with per-tool permissions and schemas; a **policy layer** (budgets, approval rules, allow-lists); **sandboxed execution** for code and browsing; **memory stores**; a **trace and replay system** so any run can be inspected and re-run; an **eval harness** with simulated environments; and **human review queues** for risky actions. The key risk controls are least privilege, step and cost limits, and human approval for irreversible steps.

14.5 From laptop to production: one project, end to end

Follow a single project through every stage to see how the modules connect. The project: *a document Q&A assistant for a company's internal policies.*

Stage 1, laptop prototype (day 1-3). Notebook with a few PDFs, a basic chunker, a local embedding model, FAISS, one prompt, a hosted model. Goal: prove the idea works on 10 questions.

Stage 2, make it a real codebase (week 1). Repository structure, locked environment, typed settings, prompts moved to files, functions extracted and unit tested, pre-commit, git. (Module 2)

Stage 3, real data pipeline (week 1-2). Layout-aware parsing, metadata, chunking, pgvector with metadata filters, incremental ingestion with document hashing, validation checks. (Module 3)

Stage 4, evaluation first (week 2). Golden set of 100 questions, retrieval metrics, groundedness judge calibrated by hand, error analysis of the first failures, improvements ordered by payoff (hybrid search, reranker, query rewriting). (Modules 5-6)

Stage 5, service and container (week 2-3). FastAPI with streaming, request ids, health checks, auth, Docker multi-stage image, docker-compose for local parity. (Module 7)

Stage 6, CI/CD and cloud (week 3-4). GitHub Actions with lint, tests, image build and eval gate; Terraform for the infrastructure; staging then production on a managed container platform; secrets in a secrets manager; canary rollout with a feature flag. (Module 9)

Stage 7, gateway, guardrails and security (week 4). LLM gateway with routing, caching, fallbacks and budgets; PII masking, injection tests, document-level access control, output validation. (Modules 11, 13)

Stage 8, observability (week 4-5). Tracing, dashboards, cost tracking, sampled online judging, feedback buttons, alerts with runbooks. (Module 10)

Stage 9, launch and learn (week 5+). Pilot with a small group, read traces daily, label failures, grow the eval set, tune cost, and then scale. Revisit build-versus-buy and self-hosting only when numbers justify it.

Notice the order: **evaluation arrives before heavy engineering,** and observability arrives before wide launch. Teams that skip either end up unable to tell whether changes help.

14.5b Common anti-patterns

- **Demo-driven development:** impressive on five hand-picked examples, never evaluated on real traffic
- **Agent for everything:** using autonomous loops where a fixed workflow would be cheaper, faster and testable

- **Prompt spaghetti:** giant prompts edited in production with no versioning or evals
- **No separation of concerns:** model calls, business logic and HTTP handling tangled in one file
- **Ignoring data quality:** tuning prompts when the real problem is bad parsing or stale documents
- **Following a moving model alias** in production and being surprised by behavior changes
- **Logging nothing, or logging everything:** no traces to debug, or raw PII in logs
- **Security by prompt:** relying on 'do not reveal secrets' instead of enforcing permissions in code
- **Premature complexity:** Kubernetes, multi-agent frameworks and fine-tuning before a simple version has been measured
- **No cost visibility** until the first shocking invoice

14.6 Build, buy or compose

For each capability (gateway, vector DB, observability, guardrails, evals, orchestration), decide whether to use a managed product, an open-source component, or build it. Build when it is core to your differentiation or constraints are unusual; buy or adopt when it is commodity. Prefer thin abstractions over deep framework lock-in: frameworks like LangChain or LlamaIndex can speed up prototypes, but know the underlying calls, and keep your core logic portable and testable.

Exercise 14: Do a full design on paper for 'an AI customer-support assistant for an e-commerce company handling 2 million conversations a month'. Follow the 10 steps in 14.1: requirements, scale estimate, approach, architecture diagram, eval plan, rollout plan, top five risks, and three tradeoffs you made. Compare it to 14.2 and note the differences.

Module 15: Governance, Feedback Loops and Team Practice

15.1 Ownership and process

Production AI is a team sport: product, engineering, data, security, legal and domain experts. Clarify who owns the model behavior, the data, the evals, the on-call and the budget. Hold regular **quality reviews** (look at real traces together), a **change-review** process for prompts and models, and blameless post-mortems.

15.2 Documentation that pays for itself

- **Architecture decision records (ADRs):** short notes on why each important choice was made
- **System card / model card:** purpose, intended use, data, known limitations, eval results
- **Runbooks:** how to diagnose and mitigate each alert
- **Eval README:** what each dataset and metric measures, and how to add cases
- **Prompt changelog:** what changed and the eval delta

15.3 Responsible AI practices

Measure fairness across user groups where relevant, be transparent that users are talking to AI, offer ways to correct or escalate, avoid over-claiming accuracy, keep humans accountable for high-stakes decisions, and set up channels for users to report problems.

15.4 The continuous improvement loop

```
Production traffic -> traces + feedback -> triage and labeling
        ^                                          |
        |                                          v
   Release (gated) <- fix (prompt, retrieval, data, model) <- grow eval set
```

Run this loop weekly. The teams that win are not those with the cleverest first version, but those with the fastest, safest loop.

15.5 Staying current

Models, prices, tools and best practices change monthly. Protect yourself by keeping abstractions thin (gateway, provider-agnostic interfaces), keeping evals strong (so you can adopt a better model in a day), and reading primary sources: provider documentation and changelogs, engineering blogs, and research papers.

Module 16: Capstones, Roadmap and Checklists

16.1 Capstone projects (choose one, build it properly)

1. **Production RAG assistant** over a real document set: hybrid retrieval with reranking, citations, access control, eval suite with CI gate, tracing, dashboards, canary deploy

2. **LLM gateway:** a multi-provider completion service with routing, fallbacks, caching, rate limits, budgets, guardrails and cost reporting
3. **Agentic workflow** (for example, a research or code-review agent): tool layer, step and cost budgets, persisted state, trajectory evals, human approval for risky actions
4. **Classical ML pipeline:** data validation, feature store, orchestrated training, registry, batch and online serving, drift monitoring, automated retraining with gates
5. **Self-hosted inference stack:** open model on vLLM, quantization comparison, load tests, autoscaling on queue depth, cost comparison against a hosted API

A strong capstone has: a README with architecture diagram and tradeoffs, one-command local setup, tests and evals in CI, a deployed instance, observability, and a short write-up of what failed and what you learned.

16.2 A 12-week learning roadmap

Weeks	Focus	Deliverable
1	Modules 1-2: lifecycle, repo structure, environments	Restructured project with locked env, tests, pre-commit
2	Module 3: data pipelines and vector search	Ingestion pipeline with versioning and validation
3	Modules 4-5: experiments, prompting, RAG, tools	Working RAG service with structured outputs
4	Module 6: evaluation	100-case golden set, retrieval and judge metrics, error analysis report
5	Module 7: containers and APIs	Streaming API in Docker with health checks
6	Module 8: serving and inference	vLLM/Ollama load-test report
7	Module 9: CI/CD and IaC	Pipeline with eval gate; deployed staging
8	Module 10: observability	Tracing, dashboards, alerts, feedback capture
9	Module 11: security and guardrails	Threat model, injection test suite, guardrails in gateway

Weeks	Focus	Deliverable
10	Modules 12-13: cost, latency, reliability	Router, caching, fallbacks, load test
11	Module 14: system design practice	Three full design write-ups
12	Module 15-16: polish and launch	Capstone deployed with documentation

16.3 Master production-readiness checklist

Code and environment

- Locked dependencies, typed config, no secrets in code, tests and linting in CI

Data

- Versioned, validated, lineage known, PII handled, freshness and deletion handled

Model and prompts

- Pinned model versions, prompts in version control, registry or release process, rollback tested

Evaluation

- Golden set, retrieval and generation metrics, calibrated judge, red-team suite, CI gate, online sampling

Serving

- Containerized, stateless, health checks, timeouts, streaming, autoscaling, load tested

Operations

- Tracing, metrics, dashboards, SLO alerts with runbooks, cost tracking, feedback loop

Security and governance

- Least privilege, permissions enforced in code, injection tests, output validation, audit logs, documented system card, incident plan

Reliability and cost

- Retries, circuit breakers, fallbacks, degradation ladder, budgets and quotas, cost per request known

16.4 Concept-to-tool cheat sheet

Need	Common options
Env and packaging	uv, Poetry, Docker, devcontainers
Config and secrets	pydantic-settings, Vault, cloud secret managers
Orchestration of data/ML jobs	Airflow, Prefect, Dagster, Kubeflow, Temporal
Data versioning and quality	DVC, lakeFS, Delta/Iceberg, Great Expectations, pandera
Feature store	Feast, Tecton, Vertex/Databricks feature stores
Experiment tracking and registry	MLflow, Weights & Biases
Vector search	pgvector, Qdrant, Weaviate, Pinecone, Milvus, OpenSearch
LLM app frameworks	Direct SDKs, LangChain/LangGraph, LlamaIndex, DSPy (use thin)
Evaluation	Ragas, DeepEval, promptfoo, Braintrust, Inspect
Observability	OpenTelemetry, Langfuse, LangSmith, Phoenix, Datadog, Grafana
Gateway	LiteLLM, Portkey, Kong, cloud gateways, in-house
Guardrails	Guardrails AI, NeMo Guardrails, Llama Guard, Presidio
Serving	vLLM, SGLang, TGI, TensorRT-LLM, Triton, KServe, BentoML, Ray Serve
CI/CD and IaC	GitHub Actions, GitLab CI, ArgoCD, Terraform, Pulumi
Runtime platforms	Cloud Run, ECS/Fargate, Kubernetes, Modal, SageMaker, Vertex AI
Load testing	Locust, k6

16.5 Recommended reading and practice

- Provider documentation and prompt-engineering guides (Anthropic, OpenAI, Google) and the 'Building effective agents' essay
- *Designing Machine Learning Systems* (Chip Huyen), *AI Engineering* (Chip Huyen), *Designing Data-Intensive Applications* (Martin Kleppmann), Google's *Site Reliability Engineering* book
- Google's MLOps maturity-level guidance and the Rules of Machine Learning
- OWASP Top 10 for LLM Applications; NIST AI Risk Management Framework
- vLLM and Hugging Face documentation for inference internals; the PagedAttention and FlashAttention papers
- Hamel Husain's writing on evals and error analysis
- Practice: contribute to an open-source LLM tool, read production post-mortems, and rebuild small versions of the reference architectures yourself

16.6 Final advice

1. Get a thin slice working end to end before perfecting any part
2. Build the eval set early and let it drive decisions
3. Instrument from day one: traces, tokens, cost
4. Prefer simple, composable designs; add complexity only when measurements demand it
5. Treat prompts, data, indexes and configs as first-class, versioned artifacts
6. Design for failure, security and cost together, not as afterthoughts
7. Keep the improvement loop short: observe, label, fix, evaluate, release

If you can explain, for any AI system, how data flows in, how quality is measured, how changes ship safely, how failures are handled, and what it costs, you understand LLMOps.

PART E: DEEP-DIVE TRACKS

Parts A-D gave you the end-to-end map. Part E goes deeper into five tracks that decide whether you can actually *run* AI systems in production. Each one extends modules you have already read:

Track	Module	Extends
1. DevOps and backend engineering (your foundation)	17	2, 7, 9
2. MLOps for traditional ML and deep learning	18	3, 4, 8, 9, 10
3. LLMOps specialization	19	5, 6, 10-13
4. Production system design and distributed systems	20	13, 14
5. Monitoring, observability and reliability	21	10, 13

Suggested order: Track 1 first, because everything else runs on top of it. Then Track 4 (design thinking) alongside Track 2 or 3, depending on whether you are building classical ML or LLM products. Track 5 grows with every project you ship.

Module 17: DevOps and Backend Engineering (Track 1)

Goal: understand how an application *actually runs on a server*, not just how to write its Dockerfile. When something breaks at 2 a.m., this knowledge is what lets you find out why.

17.1 How a program runs on Linux

Processes and threads

- A **process** is a running program with its own memory space, file descriptors and PID. A **thread** shares memory with its siblings inside one process
- Processes form a tree; PID 1 is special. In a container, *your app is often PID 1*, which means it must handle signals and reap zombie children (use `tini` or Docker's `--init`)
- States you will see in `ps` / `top`: running, sleeping, stopped, zombie. Load average and CPU vs I/O wait tell you whether you are compute-bound or waiting on something
- Essential commands: `ps aux`, `top` / `htop`, `kill`, `pgrep`, `lsof`, `strace`, `journalctl`, `dmesg`, `free -m`, `df -h`, `iostat`, `vmstat`

Signals and shutdown

- `SIGTERM` asks a process to stop gracefully; `SIGKILL` cannot be caught. Kubernetes and Docker send SIGTERM first, then SIGKILL after a grace period
- Your service must catch SIGTERM, stop accepting new requests, finish in-flight ones and close connections. Uvicorn and Gunicorn do this when configured; many hand-rolled scripts do not

Memory and limits

- Virtual vs resident memory; page cache; swap. The **OOM killer** terminates processes when memory runs out, and in containers the cgroup memory limit triggers it (exit code 137). Loading a large model into a container with too small a limit is a classic cause
- **File descriptors:** every socket and open file uses one. `ulimit -n` too low produces 'too many open files' under load
- Use `systemd` units (or a container runtime) to supervise processes, restart on failure and manage logs

Filesystem and permissions: users, groups, `chmod` / `chown`, why containers should run as non-root, read-only root filesystems, `/proc` and `/sys` as windows into the kernel.

17.2 Networking you must understand

- **IP, ports, TCP vs UDP.** A server *listens* on a port; clients connect from ephemeral ports. `0.0.0.0` means all interfaces, `127.0.0.1` only local. Binding to localhost inside a container is a very common cause of 'connection refused'
- **TCP handshake, keep-alive, timeouts, connection reuse.** Opening a new TLS connection per request is slow; reuse them with connection pools
- **DNS:** how names become IPs, TTLs, why 'it works by IP but not by name' happens, and how service discovery works in Docker and Kubernetes
- **HTTP/1.1, HTTP/2, WebSockets, SSE, gRPC:** what each is good for; streaming LLM tokens usually uses SSE
- **TLS:** certificates, termination at a load balancer or reverse proxy, mTLS between services
- **Reverse proxy and load balancer:** Nginx, Caddy, Traefik, HAProxy, cloud ALB/NLB. They terminate TLS, route by host/path, buffer or stream, enforce limits and health-check backends
- **Firewalls, security groups, VPCs, subnets, NAT:** public vs private subnets; databases and model servers belong in private subnets

- Debug tools: `curl -v`, `ss -tulpn`, `netstat`, `dig`, `nslookup`, `traceroute`, `tcpdump`, `nc`

A typical production request path:

```
Client -> DNS -> CDN/WAF -> Load balancer (TLS termination)
       -> Reverse proxy / ingress -> App container (Uvicorn workers)
       -> Postgres / Redis / queue / model API (private network)
```

If you can explain what happens at each hop and how it can fail, you understand the system.

17.3 Advanced Docker and Docker Compose

Under the hood

- Containers are ordinary Linux processes isolated with **namespaces** (PID, network, mount, user) and limited with **cgroups** (CPU, memory, I/O). They share the host kernel, unlike VMs
- Images are stacked read-only **layers** plus a thin writable layer. Layer order drives build cache and image size

Advanced practices

- BuildKit features: cache mounts (`RUN --mount=type=cache`), secret mounts so secrets never land in layers, multi-platform builds (amd64 and arm64)
- Multi-stage builds, distroless or slim runtimes, pinned digests for supply-chain safety, SBOMs and image signing
- Run with `--read-only`, drop capabilities, no `--privileged`, set `--memory` and `--cpus` limits, and set a restart policy
- **Networking modes:** bridge (default, with user-defined networks giving DNS by service name), host, none. **Volumes:** named volumes for persistent data, bind mounts for local development, tmpfs for scratch
- Inspect and debug: `docker logs`, `docker exec`, `docker inspect`, `docker stats`, `docker system df`, `docker history`

Compose for real local environments

```
services:
  api:
    build: .
    env_file: .env
```

```
    ports: ['8000:8000']
    depends_on:
      db: { condition: service_healthy }
      redis: { condition: service_started }
  db:
    image: postgres:16
    environment: { POSTGRES_PASSWORD: dev }
    volumes: ['pgdata:/var/lib/postgresql/data']
    healthcheck:
      test: ['CMD-SHELL', 'pg_isready -U postgres']
      interval: 5s
  redis:
    image: redis:7
  worker:
    build: .
    command: celery -A app.worker worker
volumes:
  pgdata:
```

Key ideas: health checks with `depends_on` conditions, profiles to toggle optional services (GPU model server, observability stack), override files for dev vs CI, and the fact that Compose is for local and small deployments, not a substitute for an orchestrator at scale.

17.4 FastAPI, asynchronous Python and concurrency

This is where most production Python AI services succeed or fail.

The mental model

- **Concurrency** is handling many things at once by interleaving; **parallelism** is doing many things at the same instant on multiple cores
- `asyncio` runs one **event loop** per process on a single thread. Code `await`s I/O, and while it waits the loop runs other tasks. Perfect for LLM apps, which spend most of their time waiting on network calls
- The **GIL** lets only one thread run Python bytecode at a time, so threads do not speed up CPU-bound Python. CPU-bound work needs processes (or native code that releases the GIL, like NumPy, PyTorch)

The cardinal sin: blocking the event loop. One synchronous call (`requests.get`, a CPU-heavy embedding on the CPU, a big `json` parse, `time.sleep`) inside an `async def` freezes *every* request on that worker. Fixes: use async clients (`httpx`, async DB drivers),

or push blocking work to a thread pool (`run_in_threadpool`, `asyncio.to_thread`) or a separate worker process or service.

Patterns you need

- `asyncio.gather` for parallel independent calls (several retrievals at once); `asyncio.timeout` / `wait_for` for deadlines; task cancellation when the client disconnects
- `asyncio.Semaphore` to cap concurrent calls to a rate-limited provider; bounded queues for backpressure
- `lifespan` handlers to create expensive shared resources (HTTP client, DB pool, loaded model) once at start-up and close them at shutdown
- **Dependency injection** (`Depends`) for settings, DB sessions, auth and clients, which also makes testing easy
- Background tasks are only for tiny fire-and-forget work; anything important or long goes to a real queue
- **Workers:** run several Uvicorn workers (via Gunicorn with the Uvicorn worker class, or one container per core under an orchestrator). Each worker is a separate process with its own memory, so a large model loaded in every worker multiplies memory use

Streaming and cancellation: use `StreamingResponse` for SSE, detect disconnects, and cancel the upstream LLM call to stop paying for tokens nobody reads.

17.5 PostgreSQL

- **Data modeling and constraints:** primary and foreign keys, unique and check constraints, JSONB for semi-structured fields, enums and timestamps with time zone
- **Indexes:** B-tree (default), GIN (JSONB, full text), partial and composite indexes, and the **pgvector** extension with HNSW/IVFFlat for embeddings. Read `EXPLAIN (ANALYZE)` plans; learn why a sequential scan happens
- **Transactions and isolation:** ACID, `READ COMMITTED` vs `REPEATABLE READ` vs `SERIALIZABLE`, row locks (`SELECT ... FOR UPDATE`, `SKIP LOCKED` for job queues), deadlocks
- **Connection management:** each Postgres connection is a process and is expensive. Use an application-side pool (SQLAlchemy) *and* a pooler (**PgBouncer** or RDS Proxy) when you have many app instances; size pools so that instances x pool size stays under `max_connections`

- **Migrations:** Alembic (or similar) in version control, backward-compatible changes (add column, backfill, then switch, then drop) so rolling deployments never see a broken schema
- **Operations:** backups and point-in-time recovery, replication and read replicas, `VACUUM` and bloat, monitoring slow queries (`pg_stat_statements`)

17.6 Redis

- Data structures: strings, hashes, lists, sets, sorted sets, streams, HyperLogLog, bitmaps; know when each fits
- **Caching patterns:** cache-aside (most common), write-through, TTLs and eviction policies (`allkeys-lru`), avoiding **cache stampedes** with request coalescing or jittered TTLs
- **Uses in AI apps:** session and conversation state, rate limiting counters, response and embedding caches, semantic cache (with Redis vector search), distributed locks, job queues, pub/sub for streaming progress
- **Durability and scale:** RDB snapshots vs AOF, replication, Sentinel, cluster mode. Redis is memory-first; do not treat it as your system of record unless you have designed for it
- Pitfalls: unbounded keys without TTLs, giant values, the blocking `KEYS` command in production (use `SCAN`)

17.7 Message queues and background processing

Tool	Nature	Typical use
Redis queues (RQ, Celery with Redis, Arq)	Simple, fast	Small to medium background jobs
RabbitMQ	Broker with routing and acks	Task queues with rich routing
Amazon SQS / Azure Service Bus	Managed queues	Reliable cloud-native jobs, DLQs built in
Kafka / Kinesis / Event Hubs	Distributed log	High-throughput event streams, replay, many consumers

Tool	Nature	Typical use
Temporal / Step Functions	Durable workflows	Long, multi-step, must-survive-crash processes (agents, pipelines)

Concepts to master: **producers, consumers, acknowledgements, visibility timeouts**, **at-least-once delivery** (so consumers must be **idempotent**), retries with backoff, **dead-letter queues**, poison messages, ordering and partitions, consumer groups, and backpressure. Pattern for AI jobs: API validates request, stores a job row, enqueues a message and returns a job id; workers process and update status; clients poll, subscribe or receive a webhook.

17.8 Authentication, authorization and API security

- **Authentication (who are you) vs authorization (what may you do).** Keep them separate in code
- **Methods:** API keys (store only hashes, support rotation, scope per client), session cookies, **JWTs** (short-lived access tokens, refresh tokens, validate signature, expiry, audience and issuer; never put secrets inside), **OAuth 2.0 and OpenID Connect** for delegated login (authorization code flow with PKCE for apps), service-to-service with mTLS or workload identity
- **Authorization models:** RBAC (roles), ABAC (attributes), and resource-level checks. Multi-tenant apps must enforce the tenant at the *data layer* (row-level security, mandatory tenant filters), not only in the route
- **API security checklist (OWASP API Top 10 mindset):** broken object-level authorization (check the caller owns the resource id), excessive data exposure (return only needed fields), rate limiting and quotas, input validation and size limits, safe CORS (explicit origins, not `*` with credentials), CSRF protection for cookie auth, security headers, TLS everywhere, secrets in a manager, dependency scanning, audit logs
- **For AI endpoints specifically:** per-user token budgets, max prompt size, abuse and scraping detection, and never exposing provider keys to clients

17.9 CI/CD with GitHub Actions or Azure DevOps

Concept	GitHub Actions	Azure DevOps Pipelines
Definition	`.github/workflows/*.yml`	`azure-pipelines.yml`
Unit of work	Job made of steps, runs on a runner	Stage > job > step on an agent
Reuse	Reusable workflows, composite actions	Templates
Secrets	Repo/environment secrets, OIDC to cloud	Variable groups, Key Vault, service connections
Approvals	Environments with required reviewers	Environments with approvals and checks

Skills to build: triggers (push, PR, tags, schedule), caching dependencies and Docker layers, matrix builds, artifacts, **environment protection and approvals**, **OIDC federation to AWS/Azure instead of long-lived cloud keys**, versioning and tagging releases, deploy jobs with rollback, and pipeline security (pin actions by SHA, least-privilege tokens, protect against secrets leaking from forks).

17.10 AWS and Azure infrastructure map

Need	AWS	Azure
VMs	EC2	Virtual Machines
Containers (serverless)	ECS Fargate, App Runner	Container Apps
Kubernetes	EKS	AKS
Functions	Lambda	Functions
Object storage	S3	Blob Storage
Managed Postgres	RDS / Aurora	Azure Database for PostgreSQL
Managed Redis	ElastiCache	Azure Cache for Redis

Need	AWS	Azure
Queues and events	SQS, SNS, EventBridge, Kinesis	Service Bus, Event Grid, Event Hubs
Secrets	Secrets Manager, SSM Parameter Store	Key Vault
Identity	IAM (roles, policies)	Entra ID, managed identities, RBAC
Networking	VPC, subnets, security groups, ALB/NLB	VNet, NSGs, Application Gateway, Load Balancer
Container registry	ECR	ACR
ML platform	SageMaker, Bedrock	Azure ML, Azure OpenAI / AI Foundry
Monitoring	CloudWatch, X-Ray	Azure Monitor, Application Insights
IaC	CloudFormation, CDK, Terraform	Bicep, ARM, Terraform

Focus on understanding **IAM/identities and networking first**; most cloud outages and security incidents start there. Practice with: a VPC with public and private subnets, an app behind a load balancer, a database that is not publicly reachable, and a role with least-privilege permissions.

Exercise 17: Deploy your API on a plain Linux VM (not a managed platform) behind Nginx: run it under systemd with Uvicorn workers, use Postgres and Redis, terminate TLS, add a Celery worker, and then deliberately break things (kill the process, fill the disk, exhaust connections, block the port) and diagnose each with the tools above.

Module 18: MLOps for Traditional ML and Deep Learning (Track 2)

Goal: manage the *entire lifecycle of a model* so that training is reproducible, artifacts are traceable, deployment is repeatable, and degradation is detected and corrected.

18.1 Tool map

Area	Tools to explore	What it gives you
Experiment tracking	MLflow, Weights & Biases	Params, metrics, artifacts, comparison of runs
Data and model versioning	DVC	Git-like versioning for large data and models, reproducible pipelines
Model registry	MLflow Model Registry	Versions, aliases/stages, lineage, approval
Model serving	BentoML, KServe	Packaging into services, scalable inference on Kubernetes
Pipeline orchestration	Airflow, Kubeflow Pipelines	Scheduled, retried, observable multi-step workflows
Monitoring	Evidently AI	Data drift, prediction drift, model quality reports
Model testing	Deepchecks	Data integrity, train/test validation, model evaluation checks

18.2 Reproducible training

Reproducibility means someone else (or you in six months) can rebuild the same model from a git commit. You need all of the following pinned and recorded:

- **Code:** git commit hash
- **Data:** immutable version (DVC hash, lakeFS commit, or snapshot id)
- **Environment:** lockfile and base image digest, CUDA and library versions
- **Config:** hyperparameters in a config file (YAML/Hydra), not hard-coded
- **Randomness:** seeds for Python, NumPy, PyTorch (and deterministic flags where feasible); know that full bit-level determinism on GPUs can be costly
- **Pipeline definition:** explicit stages with declared inputs and outputs

DVC pipeline sketch (`dvc.yaml`):

```
stages:
  prepare:
    cmd: python src/prepare.py
    deps: [src/prepare.py, data/raw]
    outs: [data/processed]
  train:
    cmd: python src/train.py --config params.yaml
    deps: [src/train.py, data/processed]
    params: [train.lr, train.epochs]
    outs: [models/model.pkl]
    metrics: [metrics.json]
```

`dvc repro` re-runs only stages whose inputs changed; `dvc push` stores large files in S3/Azure Blob while git stores small pointer files; `git checkout` plus `dvc checkout` restores any past state.

18.3 Experiment tracking and the registry with MLflow

- **Tracking:** `mlflow.log_params`, `log_metrics` (per step), `log_artifact`, autologging for common frameworks; run comparison UI; nested runs for hyperparameter searches (Optuna integrates well)
- **Model packaging:** the MLflow model format (flavors, signature, input example) makes a model loadable the same way everywhere
- **Registry:** register a model, get a new version number, attach description and tags, and point **aliases** (for example `champion`, `challenger`) or stages at versions. Deployment pipelines load 'the champion' by alias, so promoting or rolling back is *moving an alias*, with no code change
- **Lineage:** from the registered version back to the run, data version and git commit
- Weights & Biases adds richer visualization, sweeps and team collaboration; many teams use either or both

18.4 Model artifact management

- Store artifacts in object storage with immutable, content-addressed or versioned paths; never overwrite
- Record format and dependencies (pickle is unsafe to load from untrusted sources; prefer `safetensors`, ONNX or MLflow flavors)
- Keep a **model card** with intended use, training data, metrics, limits

- Sign and scan artifacts where supply-chain risk matters; apply retention rules to old versions

18.5 Deployment and serving

BentoML: define a service in Python, bundle code, model and dependencies into a **Bento**, build a Docker image, and deploy anywhere. Offers adaptive batching, async endpoints, GPU support, and runners that separate model compute from API logic.

KServe: a Kubernetes-native `InferenceService` custom resource that handles autoscaling (including scale to zero), canary rollouts by traffic percentage, multiple runtimes (sklearn, XGBoost, PyTorch, Triton, vLLM-style LLM runtimes), request/response logging and explainability hooks. Good when you already run Kubernetes and have many models.

Choosing: BentoML for developer-friendly packaging and flexible hosting; KServe when you need standardized, autoscaled, multi-model serving on Kubernetes. Whatever you pick, keep the contract (input schema, output schema, version) explicit.

Release patterns for models: shadow, canary and A/B (Module 9), with **rollback = repoint the alias or traffic split to the previous version**, which must be a one-command, rehearsed operation. Keep the previous version warm for fast rollback.

18.6 Pipeline orchestration: Airflow and Kubeflow

- **Airflow:** DAGs of tasks with schedules, retries, SLAs, backfills, sensors and a UI. Excellent for data and batch pipelines, huge ecosystem of providers. Keep DAG code light; run heavy work in containers (KubernetesPodOperator) or external systems
- **Kubeflow Pipelines:** ML-native pipelines compiled to Kubernetes workflows; each step is a container with typed inputs and outputs, with caching and experiment tracking integration. Best on Kubernetes-centric ML platforms
- Principles for both: idempotent tasks, parameterized runs, explicit dependencies, small steps, alert on failure, and store step outputs as versioned artifacts, not as shared mutable state

18.7 Testing models with Deepchecks (and friends)

Beyond accuracy, test:

- **Data integrity:** duplicates, missing values, mixed types, outliers, label issues, feature leakage

- **Train-test validation:** distribution differences, leakage between splits, label drift
- **Model evaluation:** weak segments (where does it perform worst), calibration, overfitting, performance across slices
- **Behavioral tests:** invariance (changing irrelevant details should not change output), directional expectations, minimum functionality tests, fairness across groups

Run these suites in CI as part of the training pipeline and use them as **promotion gates** before a model reaches the registry's production alias.

18.8 Monitoring with Evidently AI: data drift, model drift, performance

- **Data drift:** input feature distributions shift from the reference (training) set. Evidently provides statistical tests and distances per feature (KS test, PSI, Wasserstein, Jensen-Shannon, chi-square) and an overall drift share
- **Prediction / target drift:** the distribution of outputs or labels changes
- **Concept drift:** the relationship between features and target changes; detectable only when you get ground truth, often delayed, so also watch proxies
- **Model performance monitoring:** accuracy, precision/recall, RMSE etc. once labels arrive; slice by segment; **data quality** checks (missing values, range violations) catch upstream pipeline breakage
- Run Evidently as a scheduled job producing reports and metrics, push key numbers to Prometheus/Grafana, and alert on thresholds
- Use a sensible reference window and avoid alert noise: drift in an unimportant feature is not an incident

18.9 Automated retraining

```
Trigger (schedule | drift alert | enough new labels | performance drop)
   -> Pull versioned data -> Validate (Deepchecks / data tests)
   -> Train (tracked in MLflow) -> Evaluate vs champion on fixed + recent
data
   -> Gate: better by margin? fairness and slice checks pass?
   -> Register as challenger -> Shadow / canary in production
   -> Promote alias to champion (or auto-reject) -> Monitor -> repeat
```

Guardrails for automation: never auto-promote without comparison against the current champion, keep a human approval step for high-risk models, cap retraining frequency and cost, and always keep the ability to roll back instantly. Retraining on drifted data can also bake in errors, so validate the data first.

18.10 Versioning and rollback, summarized

- Version four things together: **code, data, model, config**, and link them through run ids
- Registry aliases make promotion and rollback an atomic metadata change
- Keep previous champion artifacts and the exact serving image available
- Rehearse rollback in staging; measure time to recover

Exercise 18: Train any tabular model with a DVC pipeline and MLflow tracking, register the best run, serve it with BentoML, then simulate drift by shifting an input feature. Use Evidently to detect it, trigger an Airflow DAG that retrains and compares against the champion, and roll back by moving the alias.

Continued: LLMOps specialization, distributed systems, monitoring stack.

Module 19: LLMOps Specialization (Track 3)

Goal: master the parts of operations that exist *only* because of LLMs, RAG and agents. Modules 5, 6 and 10-13 introduced them; this module turns each into a concrete practice and toolset.

19.1 Tool map

Purpose	Tools	Role
Observability and tracing	Langfuse, LangSmith	Traces, token and cost tracking, prompt management, datasets, feedback
Evaluation	Ragas, DeepEval	RAG and LLM metrics, test cases, CI integration
Serving	vLLM, Ollama	Self-hosted production inference (vLLM), local development (Ollama)
Guardrails	Guardrails AI	Validators for inputs and outputs, structured output enforcement
Caching	Redis	Exact and semantic caching, rate limit counters
Workflow orchestration	LangGraph	Stateful graphs for agents with checkpoints and recovery

19.2 LLM observability and tracing in practice

- Instrument with the Langfuse or LangSmith SDK (or OpenTelemetry with their exporters) so every request becomes a **trace** with nested **spans**: retrieval, rerank, each LLM generation, each tool call
- Attach metadata: user and session ids, environment, prompt name and version, model, release version, tenant. These become the filters you use to debug ('show me traces from prompt v7 with thumbs-down in the last day')
- Capture token usage and cost automatically per generation, aggregate by user, feature, prompt and model
- Attach **scores** to traces: user feedback, deterministic checks, LLM-judge results. Scores make quality queryable
- Use sessions to see whole conversations, and use traces to build datasets from real failures with one click
- Self-host Langfuse when data cannot leave your infrastructure; sanitize inputs before logging

19.3 Evaluation and benchmarking with Ragas and DeepEval

- **Ragas:** RAG-focused metrics such as faithfulness, answer relevancy, context precision, context recall, and synthetic test-set generation from your documents
- **DeepEval:** a pytest-style framework: write test cases (`input`, `actual_output`, `retrieval_context`, `expected_output`) and assert metrics (G-Eval for custom rubrics, hallucination, answer relevancy, tool correctness, task completion). Fits naturally into CI
- **Benchmarking** means comparing configurations on *your* tasks, not public leaderboards. Public benchmarks (MMLU, SWE-bench, etc.) help shortlist models; your own golden set decides
- Run an **experiment matrix**: model x prompt version x retrieval settings, record the results in Langfuse/LangSmith datasets or a table, and pick the cheapest configuration that clears your quality bar

19.4 Prompt and model versioning

- **Prompt versions:** keep prompts in git for review *or* in a prompt registry (Langfuse/LangSmith prompt management) with versions, labels (`production`, `staging`), and rollbacks without redeploying. Whichever you pick, log the version on every trace

- **Model versions:** pin exact model ids; track deprecation dates; treat a model upgrade like a release with eval comparison and canary rollout
- **Release bundle:** a release is the tuple (code version, prompt versions, model id, index version, tool and guardrail configs). Record it so any past behavior can be explained and reproduced

19.5 RAG evaluation as a workflow

1. Generate or collect questions; label relevant documents for a retrieval test set
2. Measure retrieval alone (recall@k, MRR) and tune it first; cheap and fast
3. Measure faithfulness and answer relevancy end to end with a calibrated judge
4. Slice by question type and document source; read the worst 20 failures by hand
5. Fix the biggest bucket (parsing, chunking, filters, reranker, prompt), re-run, compare
6. Freeze the suite in CI; monitor the same metrics on sampled production traffic

19.6 Token consumption and cost optimization

- Track input and output tokens per span; find the **fat prompts** (huge retrieved context, long histories, verbose tool schemas)
- Techniques: context trimming and re-ranking, summarizing history, prompt caching of static prefixes, model routing, batch APIs, response length caps, deterministic code instead of LLM calls
- Make cost a first-class metric: cost per request, per user, per successful task, plus budgets and alerts (Module 12)

19.7 Semantic caching with Redis

An exact cache hits only when the text is identical. A **semantic cache** reuses an answer when a new question is *close in meaning*:

1. Embed the incoming question
2. Run a vector similarity search over cached question embeddings (Redis vector search, RedisVL, or GPTCache-style libraries)
3. If the best match is above a **strict similarity threshold** (and the context, tenant, language and prompt version match), return the cached answer
4. Otherwise call the LLM and store the new (embedding, answer) with a TTL

Safeguards: include tenant and permission scope in the cache key (never leak answers across users), version-stamp entries so prompt or knowledge changes invalidate them, exclude personalized or time-sensitive queries, start with a high threshold, log hit quality, and evaluate false-hit rate. Semantic caching shines for FAQ-style traffic and is risky for nuanced or personalized questions.

19.8 Rate limiting and retries for LLM providers

- Providers limit **requests per minute and tokens per minute**. A request-only limiter is not enough; track estimated tokens too
- Implement a **token-bucket** limiter in Redis (shared across instances) keyed by provider, model and tenant, so one tenant cannot burn your whole quota
- Retry only on retryable errors (429, 5xx, timeouts), with **exponential backoff and jitter**, honoring `retry-after`; cap attempts and total deadline; do not retry on 4xx validation errors
- Use a queue with workers for bursty workloads, and prioritize interactive traffic over batch
- Apply **per-tenant budgets** and soft/hard quotas with clear errors

19.9 Guardrails with Guardrails AI

- Define **validators** (regex, PII detection, toxicity, competitor mentions, valid JSON against a schema, reading level, grounding checks) and attach them to inputs or outputs through a `Guard`
- Choose an **on-fail action**: reask the model with the error, fix automatically, filter, refuse, or raise
- Combine with Pydantic models for structured outputs, so malformed output is retried with feedback before it reaches your code
- Run cheap validators first, expensive model-based ones only when needed; measure the latency each adds; test validators on your eval set for false positives and negatives
- Treat guardrails as one layer in the defense-in-depth stack from Module 11, not the whole defense

19.10 LLM provider fallbacks

- Put provider calls behind one interface (gateway or a thin client), configure a **fallback chain** (primary model, secondary model on another provider, then a cached or degraded response)
- Trigger fallbacks on errors, timeouts or latency budget breaches; combine with **circuit breakers** so a failing provider is skipped for a cool-down
- Keep prompts and structured-output schemas tested against the fallback model, and track the fallback rate and its quality separately
- Record which model actually answered in the trace

19.11 Agent reliability and workflow recovery with LangGraph

Agents are long, stateful, failure-prone processes. LangGraph models them as **graphs** of nodes (steps) and edges (control flow) over a shared **state**.

- **Checkpointing:** persist state after each step (Postgres or Redis checkpointer, keyed by a thread id). If the process crashes or a deploy happens, the run **resumes from the last checkpoint** instead of starting over
- **Human-in-the-loop:** `interrupt` before sensitive steps, wait for approval (hours or days), then continue
- **Time travel and replay:** inspect or rewind to an earlier checkpoint to debug a bad decision
- **Reliability controls:** maximum steps (recursion limit), per-node timeouts and retries, token and cost budgets, loop detection, validation of tool arguments, and safe defaults on tool errors
- **Idempotent tools:** because a step may be retried after a crash, side-effecting tools need idempotency keys so a retry does not send two emails or charge twice
- **Evaluate trajectories,** not just final answers; run each test several times and report pass rates (Module 6.5)
- For workflows that must survive for days and coordinate many systems, durable engines like Temporal complement or replace agent-framework checkpointing

19.12 vLLM and Ollama in the workflow

- **Ollama** for developers: one command to pull and run quantized open models locally, OpenAI-compatible endpoint, ideal for offline prototyping and cheap experiments

- **vLLM** for production: high-throughput batching, PagedAttention, prefix caching, multi-GPU, LoRA adapters, OpenAI-compatible server. Deploy it in a container behind your gateway, autoscale on queue depth, and monitor KV cache usage and queue length (Module 8)
- Because both speak an OpenAI-style API, application code can switch between local, self-hosted and hosted providers by config

Exercise 19: Extend your RAG service with: Langfuse tracing and prompt versioning; a DeepEval suite in CI; a Redis semantic cache with tenant-scoped keys; a Redis token-bucket limiter with backoff; Guardrails AI validators on output; a provider fallback chain; and wrap the workflow in LangGraph with a Postgres checkpointer. Kill the process mid-run and confirm that it resumes.

Module 20: Production System Design and Distributed Systems (Track 4)

This is the area most AI tutorials skip, and where strong engineers separate themselves. Invest serious time here. Every concept below shows up in real incidents and in design interviews.

20.1 Scaling

- **Vertical scaling (scale up):** bigger machine. Simple, has a ceiling, and a single machine is a single point of failure
- **Horizontal scaling (scale out):** more machines behind a load balancer. Requires **stateless services** (state in DB, cache or queue), shared nothing between instances, and idempotent operations
- **Scaling the data layer is harder:** read replicas for read-heavy loads, **sharding/partitioning** for write-heavy or huge data, caching to shield the database
- **Autoscaling** reacts to metrics (CPU, requests per instance, queue depth, GPU utilization). Set minimums for warm capacity, maximums for cost safety, and understand start-up time, since GPU and model-loading cold starts can take minutes
- Find the **bottleneck** first (measure), then scale that. Scaling the API does nothing if the database or the model provider quota is the limit

20.2 Load balancing

- **L4** (TCP level, fast, simple) vs **L7** (HTTP level, routes by path/header, can do TLS termination, retries, canary splits)
- **Algorithms:** round robin, least connections, weighted, consistent hashing (for cache affinity or sticky sessions)
- **Health checks** remove bad instances; **connection draining** lets in-flight requests finish during deploys
- For LLM serving, plain round robin can be poor because requests have very different lengths. Prefer least-outstanding-requests, queue-aware or **KV-cache and prefix-aware routing** (send requests sharing a prefix to the same replica)
- Tools: Nginx, HAProxy, Envoy, Traefik, cloud ALB/NLB/Application Gateway, Kubernetes Services and Ingress

20.3 API gateways

A gateway is the front door for APIs. It handles authentication, authorization, rate limiting and quotas, request validation, routing and versioning, TLS, caching, request transformation, and logging. Examples: Kong, Envoy-based gateways, AWS API Gateway, Azure API Management, Apigee, Traefik. In AI systems you often have **two layers**: a classic API gateway at the edge for clients, and an **LLM gateway** internally in front of model providers. Keep them distinct in your design.

20.4 Message queues, async processing and distributed workers

- Queues **decouple** producers from consumers, smooth traffic spikes, enable retries and let you scale workers independently of the API
- **Competing consumers:** many identical workers pull from one queue; add workers to increase throughput
- **Worker design:** idempotent handlers, visibility timeouts longer than the job time, heartbeats for long jobs, graceful shutdown, concurrency limits per worker, separate queues by priority and by job type (GPU jobs vs CPU jobs)
- **Backpressure:** bound queue size, reject or slow producers when full, and alert on queue age, not just length
- **Patterns:** work queue, fan-out/fan-in, pub/sub, outbox pattern (write the event in the same DB transaction as the data, then publish reliably), **saga** (a sequence of local transactions with compensating actions instead of distributed transactions)

- **Exactly-once is mostly a myth:** design for at-least-once delivery plus idempotent processing

20.5 Database connection pooling

- Opening connections is slow, and databases support limited concurrent connections. Pools reuse them
- Size pools by the arithmetic: `instances x workers x pool_size <= database max connections` (leaving headroom for admin, migrations and replicas)
- Add an external pooler (PgBouncer, RDS Proxy) for many short-lived clients or serverless; learn transaction vs session pooling and which features it breaks (prepared statements, session variables)
- Symptoms of pool problems: requests hanging waiting for a connection, 'too many connections' errors, latency spikes under load. Set pool timeouts and monitor pool saturation
- The same pooling idea applies to HTTP clients calling model providers: reuse connections, set limits and timeouts

20.6 Caching strategies

Strategy	How it works	Watch out for
Cache-aside	App checks cache, on miss reads source and fills cache	Stale data, stampedes
Read-through	Cache layer loads from source itself	Complexity of the cache layer
Write-through	Writes go to cache and source together	Write latency
Write-behind	Write to cache, flush to source later	Data loss risk
TTL and invalidation	Expire by time or explicit events	'The two hard problems': invalidation

Layers: browser and CDN, API gateway, application memory, Redis, database buffer cache. Prevent **stampedes** (many requests rebuilding the same expired key) with locking, request coalescing, jittered TTLs or serve-stale-while-revalidate. Cache what is expensive and reused; measure hit rate.

20.7 Fault tolerance, circuit breakers, retries and idempotency

- **Failure is normal:** disks, nodes, networks, dependencies and whole zones fail. Design so that the failure of one part does not take down the whole
- **Timeouts** everywhere; **retries** with exponential backoff and jitter and a **retry budget** (an uncontrolled retry storm can turn a small blip into an outage)
- **Circuit breaker** states: closed (normal), open (fail fast), half-open (probe for recovery)
- **Bulkheads:** separate pools and queues so one noisy dependency cannot exhaust shared resources
- **Idempotency:** the same request repeated has the same effect as once. Implement with idempotency keys stored with results, natural keys and upserts, and conditional writes. Essential with at-least-once queues and client retries
- **Graceful degradation, load shedding** and **timeouts that shrink as a request moves deeper** (deadline propagation) keep systems alive under overload

20.8 High availability

- Remove single points of failure: multiple instances across **availability zones**, redundant load balancers, replicated databases with automatic **failover**, replicated queues and caches
- **Active-active** (all regions serve traffic) vs **active-passive** (standby takes over). Multi-region adds big complexity, so use it only when requirements demand it
- Know **RTO** (how long you can be down) and **RPO** (how much data you can lose) and design backups and replication to meet them. Test failover, do not just configure it
- Availability math: 99.9% allows about 8.8 hours of downtime a year, 99.99% about 53 minutes. Each dependency in series lowers the total; redundancy in parallel raises it
- Remember that your availability is bounded by your model provider's unless you add fallbacks

20.9 Rate limiting

Algorithm	Idea	Notes
Fixed window	Count per time window	Simple, allows bursts at window edges

Algorithm	Idea	Notes
Sliding window	Rolling count	Smoother, more memory
Token bucket	Tokens refill at a rate, each request spends tokens	Allows controlled bursts; the common choice
Leaky bucket	Requests drain at a constant rate	Smooths traffic

Implement in the gateway or with Redis (atomic Lua scripts) so limits hold across instances. Limit by user, API key, tenant, IP and endpoint. Return `429` with `Retry-After`. For LLM apps, limit by **tokens and cost**, not just requests.

20.10 Microservices vs monolith

- **Monolith:** one deployable unit. Simple to build, test, deploy and debug; scales as a whole. Best for small teams and early products
- **Microservices:** independently deployable services with their own data. Enable team autonomy and independent scaling (for example scaling a GPU embedding service separately from the API), but add network failures, distributed tracing needs, data consistency problems and operational overhead
- **Modular monolith:** one deployable with strict internal module boundaries. Often the best starting point; split out services only when a clear need appears (different scaling profile, different team, different runtime such as a GPU model server, different release cadence)
- AI systems naturally split at: model inference services, ingestion pipelines, the main API, and background workers
- Avoid the **distributed monolith**: many services that must be deployed together and share a database

20.11 Consistency and distributed-systems concepts to know

- **CAP theorem** in practice: during a network partition you choose between consistency and availability. **Eventual consistency** is acceptable for many AI features (search indexes, caches) but not for payments or permissions
- **Replication lag** and read-your-writes problems with read replicas
- **Distributed locks** and leader election (and why they are subtle); **clock skew**; ordering

- **Consensus** exists (Raft, Paxos) inside etcd, ZooKeeper, Kafka controllers; you mostly use it through those tools
- Knowing *why* these problems exist helps you pick managed services that solve them for you

20.12 Putting it together: a scaling story

A RAG chatbot grows from 10 to 10,000 requests per minute. Week 1: single container, SQLite-like simplicity. Then Postgres, then multiple API instances behind a load balancer with sessions in Redis. Then provider rate limits hit, so you add a token-bucket limiter, a queue for non-interactive jobs and a gateway with fallbacks. Then the database connections saturate, so PgBouncer and read replicas. Then cost explodes, so caching, routing and prompt caching. Then reliability matters, so multi-AZ, circuit breakers, SLOs and on-call. At each step the move was driven by a *measured bottleneck*, which is how you should always scale.

20.13 Resources for this track

- **System Design Primer** (GitHub): scalability, caching, databases, queues, consistency, with practice problems
- **ByteByteGo:** clear diagrams and articles on system design; a good mix of free and paid content
- **Google Site Reliability Engineering book** (free online): SLOs, error budgets, monitoring, incident response, cascading failures. Also the companion SRE Workbook
- *Designing Data-Intensive Applications* by Martin Kleppmann, for the deepest treatment of storage, replication, partitioning and streams
- Practice: pick a system (URL shortener, chat app, rate limiter, notification service, then 'ChatGPT-like service' and 'enterprise RAG') and design it on paper in 45 minutes, covering requirements, estimates, architecture, bottlenecks, failure modes and tradeoffs

Exercise 20: Design and then build a small distributed version of your service: two API replicas behind Nginx, Redis for sessions and rate limiting, a queue with two workers for ingestion jobs, Postgres with PgBouncer. Load test it with k6 or Locust, find the first bottleneck, fix it, repeat three times, and write down what you measured at each step.

Module 21: Monitoring, Observability and Reliability (Track 5)

Goal: a production AI application must *tell you when something is wrong*, before your users do. Module 10 covered AI-specific signals; this module builds the general-purpose monitoring stack underneath them.

21.1 Structured logging

- Log as **JSON**, one event per line, to stdout (containers collect it). Fields: timestamp, level, service, environment, version, **trace id and request id**, user/tenant (pseudonymized), event name, duration, error type
- Use log levels consistently (DEBUG, INFO, WARNING, ERROR); never log secrets or raw PII; sample noisy debug logs
- Libraries: `structlog` or the standard `logging` with a JSON formatter; propagate context (request id) with context variables
- Logs answer 'what happened in this specific case'; they are expensive to store and search at scale, so keep retention policies sensible

21.2 Metrics with Prometheus and dashboards with Grafana

Prometheus scrapes metrics from `/metrics` endpoints on a schedule and stores time series. Metric types:

- **Counter:** only goes up (requests, errors, tokens). Query as a rate
- **Gauge:** goes up and down (queue length, in-flight requests, GPU memory)
- **Histogram:** distribution of values in buckets (request latency, tokens per request); use to compute percentiles
- **Summary:** client-side quantiles (less common)

Instrument with `prometheus-client` (or the OpenTelemetry metrics SDK) and add labels carefully: **high-cardinality labels** (user id, request id) blow up storage, so use bounded labels like route, status code, model, tenant tier.

Query examples (PromQL):

- Request rate: `sum(rate(http_requests_total[5m])) by (route)`
- Error ratio: `sum(rate(http_requests_total{status=~'5..'}[5m])) / sum(rate(http_requests_total[5m]))`
- p95 latency: `histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le, route))`

Methods for choosing metrics:

- **RED** for services: Rate, Errors, Duration
- **USE** for resources: Utilization, Saturation, Errors
- Google's **four golden signals:** latency, traffic, errors, saturation

Grafana builds dashboards over Prometheus, Loki, Tempo, Elasticsearch and more. Design dashboards for a purpose (on-call overview, per-service, cost, model quality), put the most important signals at the top, show SLO status, and link panels to traces and logs. Version dashboards as code (JSON/Terraform) where you can.

21.3 Distributed tracing with OpenTelemetry

- **OpenTelemetry (OTel)** is the vendor-neutral standard for traces, metrics and logs: SDKs and auto-instrumentation for FastAPI, HTTP clients, databases, Redis and queues; and the **Collector** which receives, processes (batching, sampling, redaction) and exports telemetry to any backend
- A trace is a tree of **spans** linked by a trace id propagated through HTTP headers (W3C `traceparent`) and message metadata, so one user request can be followed across API, queue, worker and model call
- **Sampling:** head sampling (decide at the start) is cheap; tail sampling (keep all error and slow traces) is more useful. Always keep traces for failures
- Backends: Jaeger, Grafana Tempo, Zipkin, Datadog, Honeycomb, and for LLM-specific views, Langfuse and others

21.4 The ELK Stack for logs

- **Elasticsearch** stores and indexes logs; **Logstash** (or lighter shippers such as Filebeat and Fluent Bit/Fluentd) collects and transforms; **Kibana** searches and visualizes
- Learn index lifecycle management (hot, warm, delete tiers), mappings, structured fields, and how to control cost with retention and sampling
- Alternatives worth knowing: Grafana Loki (cheaper, label-based), OpenSearch, and cloud-native tools (CloudWatch Logs, Azure Monitor/Log Analytics)

21.5 Error tracking

Tools like **Sentry** group exceptions by fingerprint, attach stack traces, release versions, user context and breadcrumbs, and notify the right people. Tie errors to releases so you can see 'this error started in v1.4.2'. Track non-exception failures too, such as repeated

schema-validation failures from an LLM.

21.6 Infrastructure monitoring

- Hosts and containers: **node_exporter**, **cAdvisor**, kube-state-metrics, the Kubernetes metrics server: CPU, memory, disk, network, restarts, OOM kills, pod pending states
- GPUs: **NVIDIA DCGM exporter** for utilization, memory, temperature, errors
- Managed services: database connections, replication lag, cache hit ratio, queue depth and age, provider quota usage (CloudWatch, Azure Monitor)
- Synthetic checks and uptime probes from outside your network (a real request through the whole stack on a schedule)

21.7 Health checks

- **Liveness:** is the process fundamentally alive? Failing it restarts the container. Keep it simple and dependency-free
- **Readiness:** can this instance serve traffic right now (model loaded, DB reachable)? Failing it removes the instance from the load balancer without restarting
- **Startup probes** for slow-loading models so the orchestrator does not kill them mid-load
- Do not make liveness depend on external services, or an outage of the database will restart your whole fleet in a loop

21.8 Alerting

- Alert on **symptoms** (user-visible: error rate, latency, failed jobs, quality score) more than causes (CPU high)
- Use **SLO-based alerts with burn rates**: page when the error budget is being consumed quickly, ticket when slowly
- **Prometheus Alertmanager** (or Grafana alerting, PagerDuty, Opsgenie) handles routing, grouping, deduplication, silencing and escalation
- Every alert needs an owner, a severity, a runbook link and a clear action. If an alert fires and nobody acts, delete or fix it
- Include AI signals: token and cost spikes, fallback rate, guardrail trigger rate, queue age, drift alerts, judge-score drops

21.9 Incident management

1. **Detect:** alert, user report or synthetic check
2. **Triage and declare:** assign severity and an **incident commander**; open a channel and a timeline doc
3. **Mitigate first, diagnose second:** roll back, flip the feature flag, fail over, shed load, switch provider. Restore service before finding the root cause
4. **Communicate:** regular updates to stakeholders and a status page for users
5. **Resolve and verify** with metrics
6. **Blameless post-mortem:** timeline, impact, root causes (plural), what went well, what went badly, and **action items with owners and dates**; add a test, eval, alert or runbook so the same failure is caught next time

Track **MTTD** (time to detect), **MTTR** (time to recover), incident count by cause, and error-budget burn. Practice with game days and failure injection in staging.

21.10 A reference observability stack

```
App (FastAPI, workers, model servers)
  |-- structured JSON logs ----> Fluent Bit -> Elasticsearch/Loki ->
Kibana/Grafana
  |-- /metrics (Prometheus) ---> Prometheus -> Alertmanager ->
PagerDuty/Slack
  |                                    \-> Grafana dashboards
  |-- OTel traces ------------> OTel Collector -> Tempo/Jaeger
  |-- LLM traces, scores, cost -> Langfuse / LangSmith
  |-- exceptions --------------> Sentry
  '-- infra exporters (node, cAdvisor, DCGM, DB, Redis) -> Prometheus
```

Start small: structured logs, a few RED metrics, health checks, one dashboard and three alerts. Add tracing and the rest as the system and team grow.

Exercise 21: Add `/metrics` with request, latency histogram, token and queue metrics to your service; run Prometheus, Grafana and an OTel Collector with Tempo in docker-compose; build an on-call dashboard; define an SLO with a burn-rate alert; then trigger a fault and walk through detection, mitigation and a written post-mortem.

Module 22: Learning Path Across the Five Tracks

Use this combined plan if you want to cover the new tracks in a structured way. Each phase ends with something you can demonstrate.

Phase	Weeks	Focus	Proof of skill
1. Server fundamentals	1-3	Module 17: Linux, networking, Docker, async FastAPI	API running on a VM behind Nginx under systemd, and diagnosed under induced failures
2. State and plumbing	4-6	Module 17: Postgres, Redis, queues, auth	Auth-protected API with pooled Postgres, Redis cache and rate limiter, queue workers
3. CI/CD and cloud	7-8	Module 17: GitHub Actions or Azure DevOps, AWS or Azure	Pipeline with OIDC deploy to a cloud environment, infra as code
4. Classical MLOps	9-11	Module 18	DVC + MLflow + BentoML/KServe + Airflow retraining with Evidently drift detection
5. LLMOps	12-14	Module 19	RAG or agent service with Langfuse, DeepEval/Ragas in CI, semantic cache, guardrails, fallbacks, LangGraph recovery
6. Distributed design	15-17	Module 20	Multi-replica deployment under load test; three written system designs
7. Observability and reliability	18-20	Module 21	Full monitoring stack, SLO alerts, runbooks, a practiced incident and post-mortem
8. Capstone	21-24	Modules 14 and 16	One production-grade project that combines everything, deployed and documented

When time is short, prioritize in this order: **(1) backend and DevOps fundamentals, (2) system design and distributed systems, (3) observability and reliability, (4) LLMOps or MLOps depending on your product, (5) cloud specifics.** Strong fundamentals make

every tool in this course easier to pick up, and tools change faster than fundamentals do.